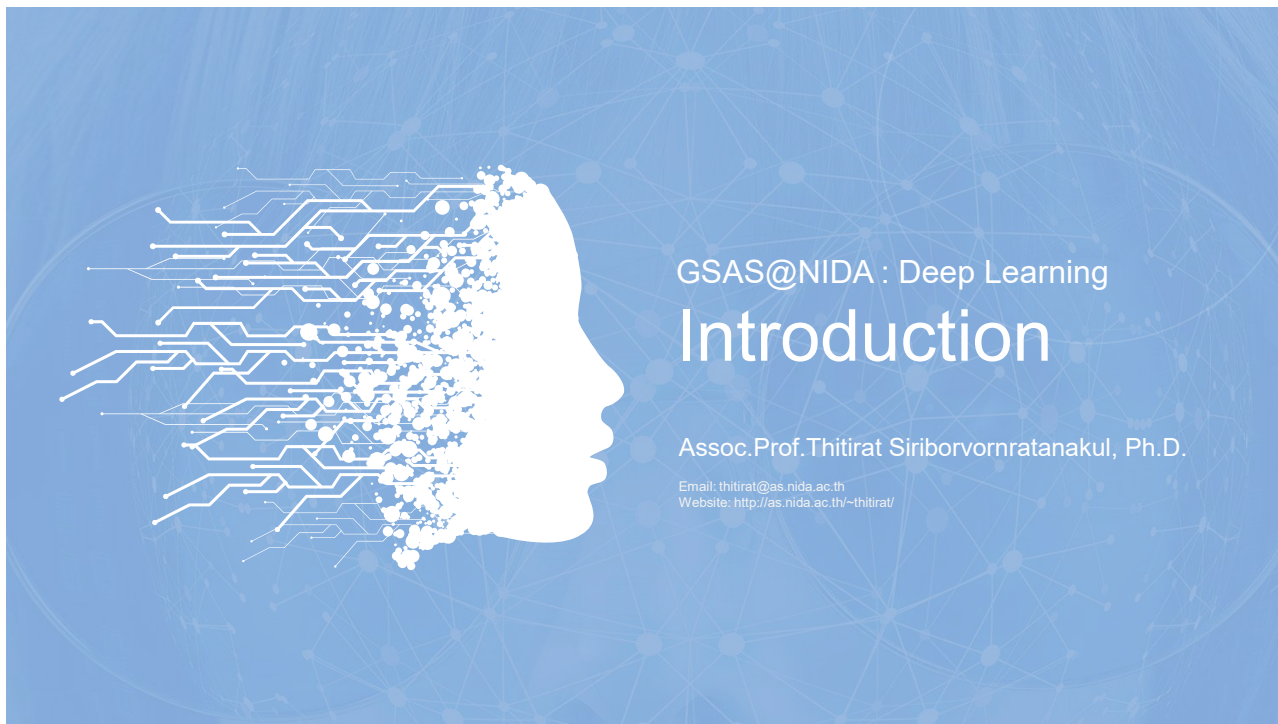




1

OUTLINE

01 Class's Introduction

Schedule, evaluation, books, and FAQs

02 History of Modern DL

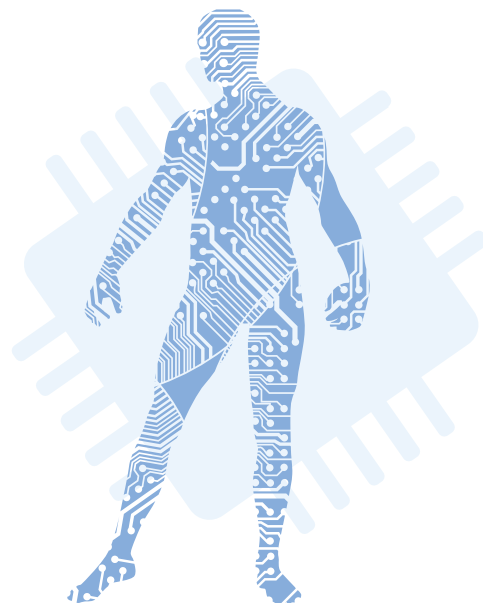
The rapid growth of Deep Learning in the modern era

03 Hardware Acceleration

Prepare necessary hardware to accelerate our deep learning project

04 DL Frameworks

Which frameworks to use in our deep learning project



2



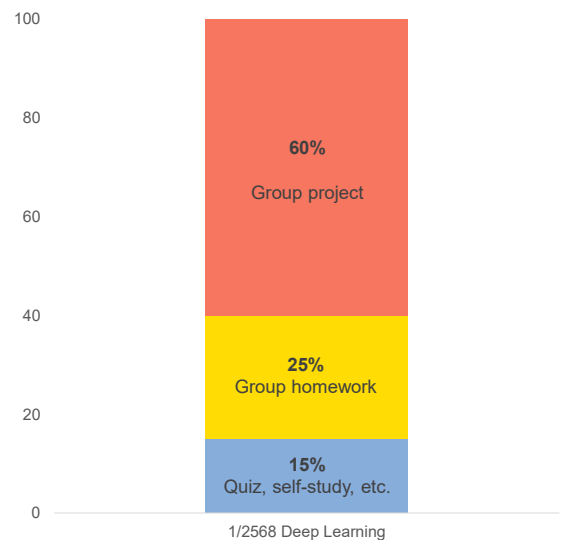
Class's Introduction

Schedule, evaluation, books, and FAQs

3

Schedule and Evaluation

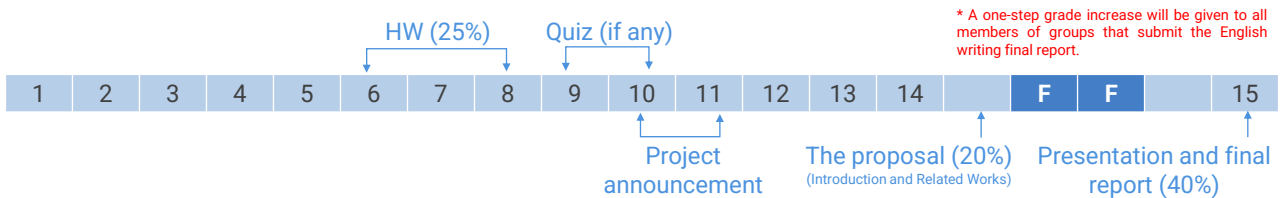
- **Subject:**
 - DADS 7202: Deep Learning (3 credits)
 - CI 7310: Deep Learning (3 credits)
- **Class material and schedule:**
 - MS Teams > General channel > Files tab > เอกสารประกอบของคลาส folder
- **Grading policy:**
 - For students with "Audit" registration, fulfilling the 80% class attendance does not guarantee the "pass" grade.
- **Important note!!!**
 - Participation in the class is only permitted for students who have completed the registration process.
 - Video recording is not allowed.



4

Schedule and Evaluation

- **15% of Quiz, self-study assignment, or other in-class activities**
- **25% of Group Homework:**
 - There is one group programming homework.
- **60% of Group Project:**
 - 20% for the proposal (Introduction and Related Works)
 - 40% for presentation and the final report*



* A one-step grade increase will be given to all members of groups that submit the English writing final report.

5

Recommended Materials



6

FAQ 1: Programming skill



7

FAQ 2: Math skill

Quora Home Answer Spaces Notifications Search C

Learning Machine Learning +6

I do not have strong mathematics background, what should I learn in mathematics to be able to master Machine Learning and AI?

Answer Follow 253 Request 1

[14SEP2017] <https://www.quora.com/I-do-not-have-strong-mathematics-background-what-should-I-learn-in-mathematics-to-be-able-to-master-Machine-Learning-and-AI>

Quora Home Answer Spaces Notifications Search C

Andrew Ng, Co-founder of Coursera; Adjunct Professor of Stanford
Answered Sep 14, 2017 · Featured on Quora Sessions's Twitter · Upvoted by Ozan Ozyegir, PhD Machine Learning, Ryerson University (2022) and Mir Junaid, Ph.D. Big Data & Machine Learning, University of Technology of Troyes (2020)

Originally Answered: I do not hold strong maths background, what all should I learn in Maths to be able to be master in Machine Learning and AI?

I think the most important areas of math for machine learning are, in decreasing order:

1. Linear algebra
2. Probability and statistics
3. Calculus (including multivariate calculus)
4. Optimization

After that, I think it falls off quickly. I've also found Information Theory helpful. You can find courses on all of these on Coursera or at most universities.

While it's hard to argue against knowing more math, I think the level of math needed to do machine learning effectively, or to get a PhD in machine learning, has decreased over the years. This is because machine learning has become more empirical (based on experiments) and less theoretical, especially with the rise of deep learning.

As a PhD student I had loved real analysis, and also studied differential geometry, measure theory, and algebraic geometry. While you're certainly be better off knowing these areas than not, in a world in which you have limited time, consider just spending more time studying machine learning itself, and even studying some of the other technical foundations for building AI systems, such as the algorithms that underly building big data systems and how to organize giant databases, plus HPC (high performance computing).

Best of luck!

400.6k views · View Upvoters · View Sharers · Answer requested by Sanjay M S and Vijendr Gaorh

Upvote 9.6k Share 73

8



FAQ 3: Prerequisite



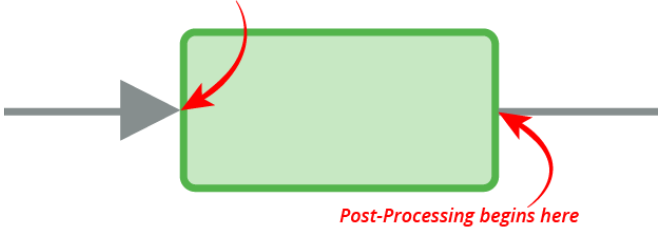
NumPy



Image generated from <https://imgflip.com/meme/generator/179755907/Skipping-steps>

9

Pre-Processing begins here



Post-Processing begins here

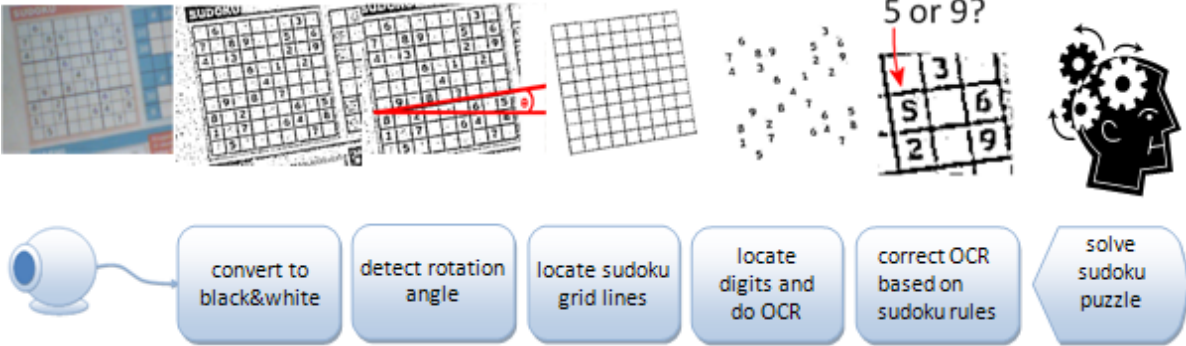


Image source: <https://www.codeproject.com/Articles/238114/Realtime-Webcam-Sudoku-Solver>

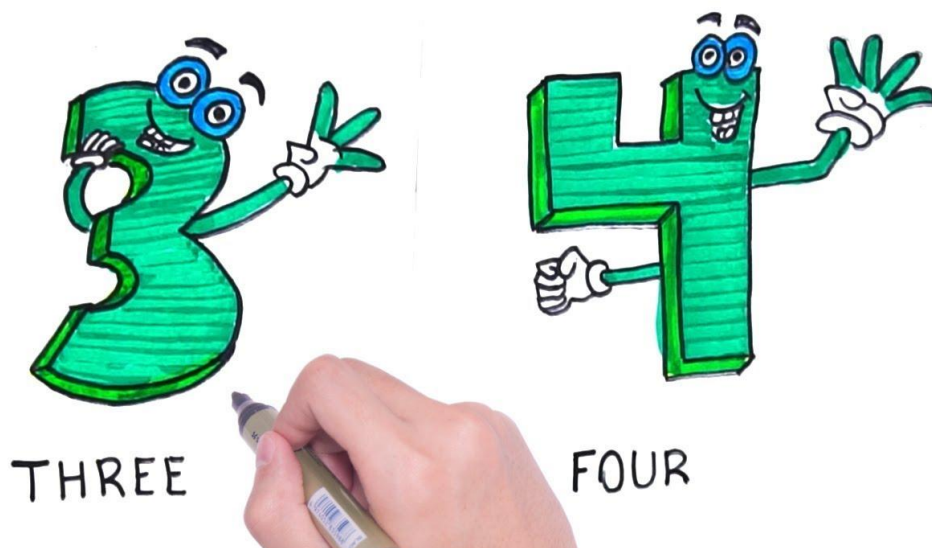
10

FAQ 4: Tools and devices



11

FAQ 5: Group members



12

Any question?



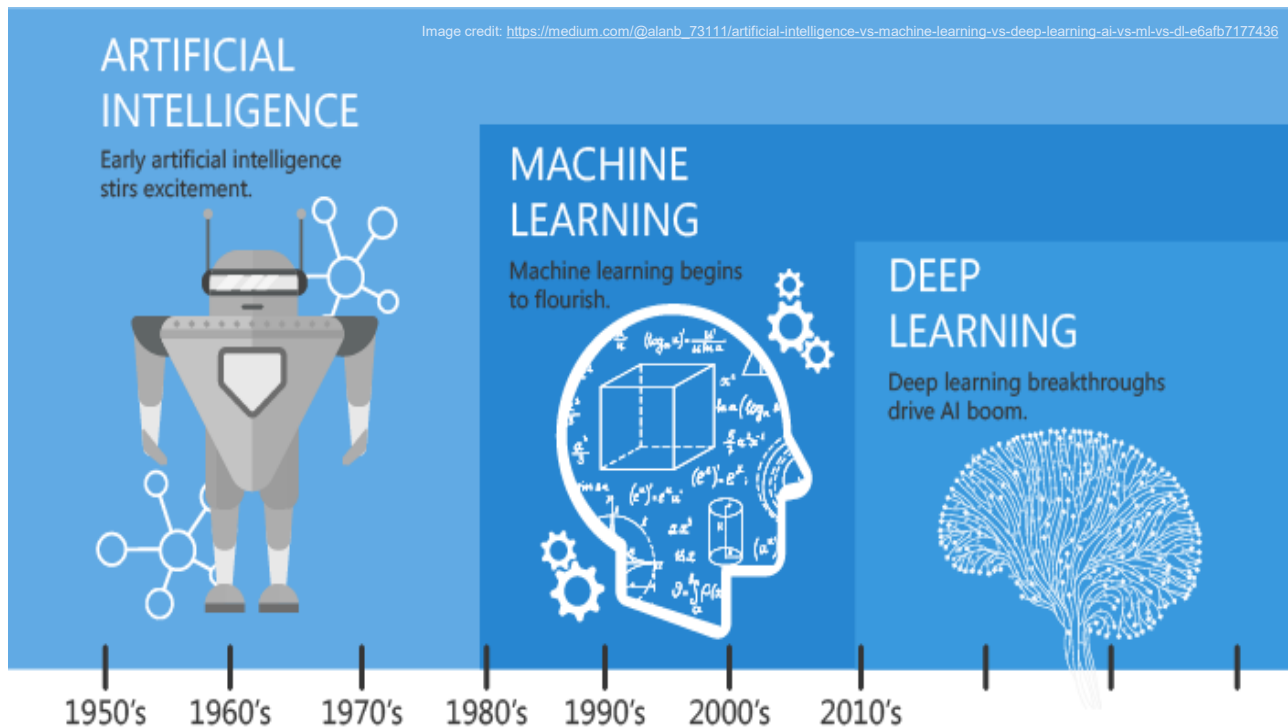
13



History of Modern DL

The rapid growth of Deep Learning
in the modern era

14



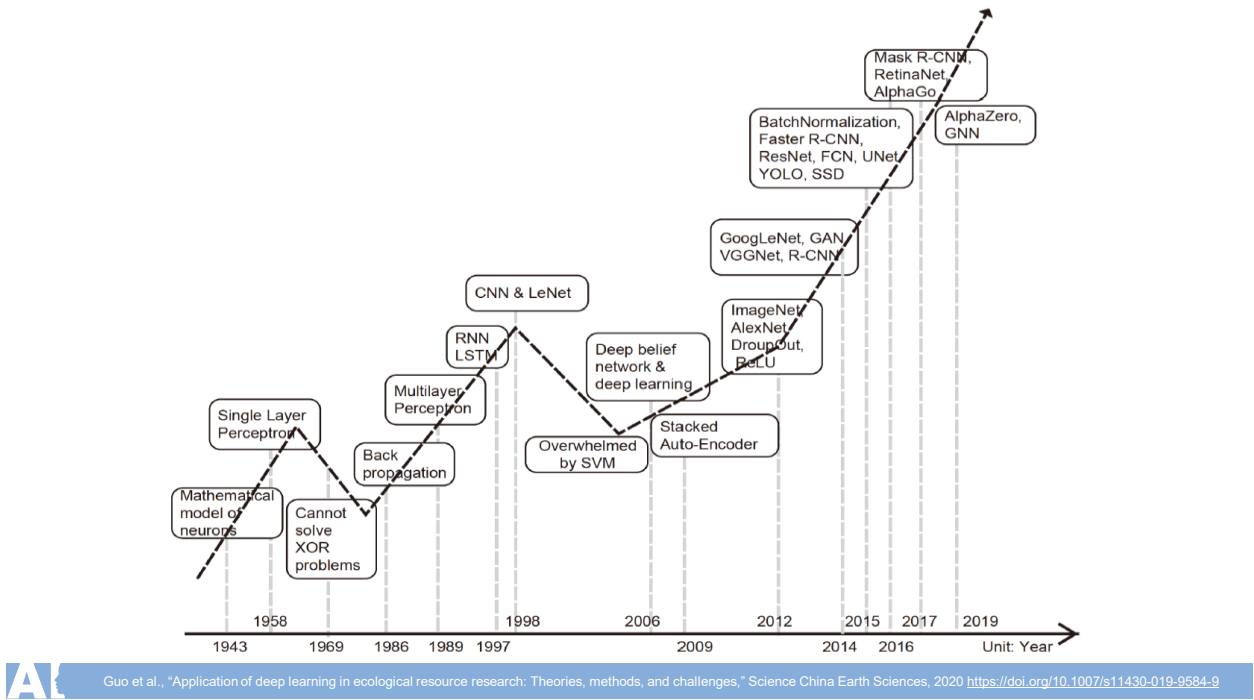
15

The universal approximation theorem

Feed-forward neural networks with at least one hidden layer can approximate any continuous function.

Papers that prove the theorem <https://ar.stackexchange.com/questions/13317/where-can-i-find-the-proof-of-the-universal-approximation-theorem>
A visual proof that neural nets can compute any function <http://neuralnetworksanddeeplearning.com/chap4.html>

16



17

THE OLD HISTORY

- **1943:** The first **neural network** was proposed by McCulloch and Pitts.
 - **1955:** John McCarthy first coined the term **Artificial Intelligence**.
 - **1959:** Samuel coined and popularized the term **Machine Learning**.
-
- **1982:** Birth of **Hopfield Network**
 - **1986:** Birth of **backpropagation**
 - **1986:** Birth of **RNN** and **AE**
 - **1989:** Birth of **CNN**
 - **1997:** Birth of **LSTM**

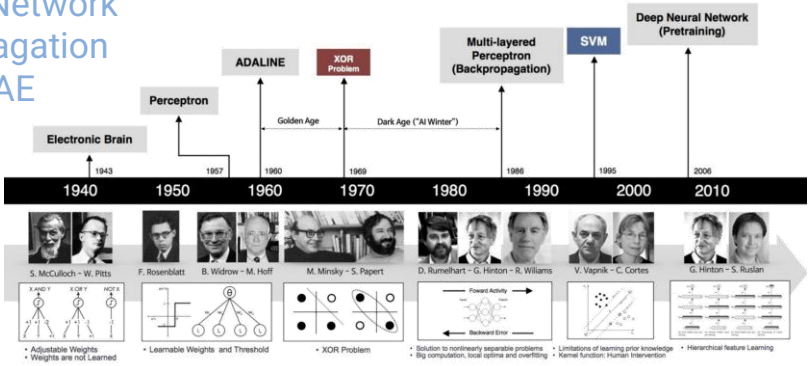


Image credit:
https://beamandrew.github.io/deeplearning/2017/02/23/deep_learning_101_part1.html

18

AI THE MODERN HISTORY

- **2006:** The **pretraining** technique by Hinton et al.



- **2012:** ImageNet evolution—**AlexNet** 

- **2014:** Birth of **GRU**, **VAE**, and **GAN**

- **2015:** Birth of **diffusion model**

- **2017:** The **DeepFake** viral

- **2017:** Birth of the groundbreaking **Transformer**



- **2018:** NLP's ImageNet moment—**BERT**

- **2018:** ACM Turing Award to Deep NN



19

AI THE MODERN HISTORY

- **2020:** Birth of **ViT**



- **2020:** **AlphaFold2** "This will change medicine. It will change research. It will change bioengineering. It will change everything."



- **2020:** **Self-supervised learning** trend in vision



- **2021:** Bridging the gap between vision and NLP—**DALL-E** and **CLIP**



- **2021:** The comeback of **diffusion models** in image generation

- **2022:** The year of Generative AI (massive adoption ):
Midjourney (July), **Stable Diffusion** (August), **ChatGPT** (November)



- **2023:** Generative AI's breakout year

- **2023:** The homerun year for **LLMs**

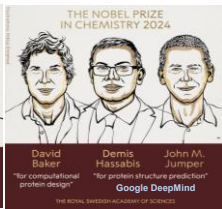
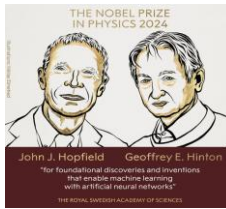


20

AI THE MODERN HISTORY

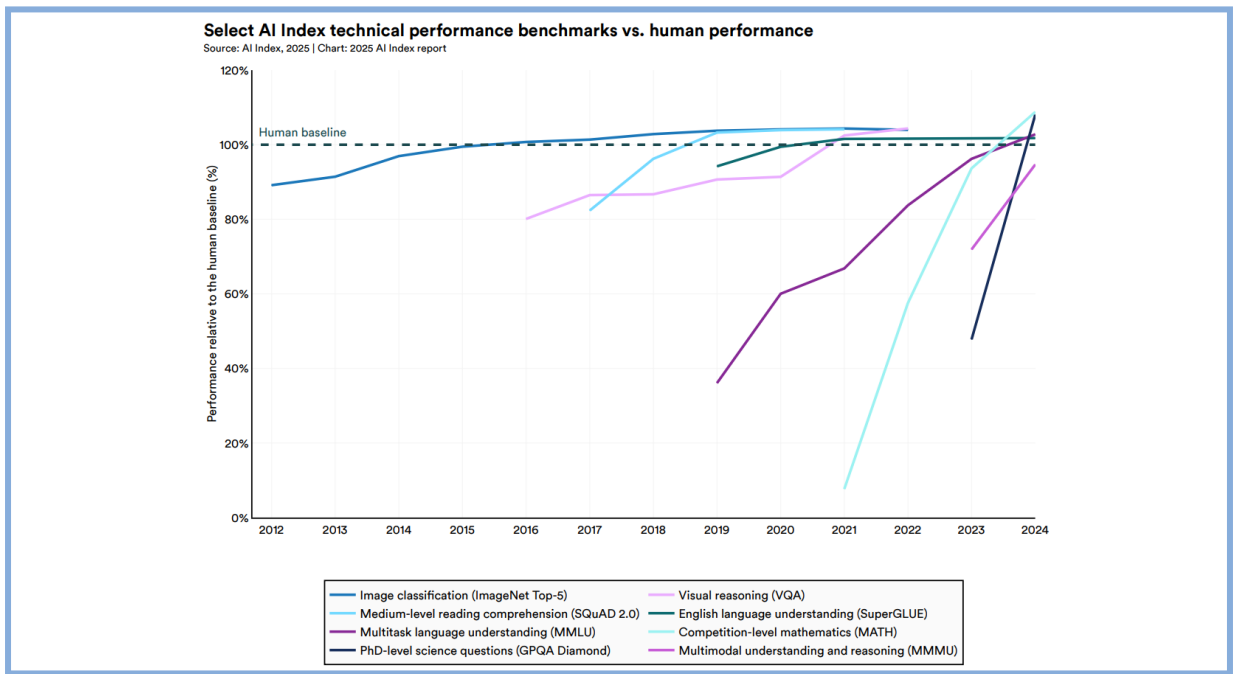


- **2024:** Multimodal LLMs, Agentic AI 🔥, Small Language Model, AI price war, Large Reasoning Model (LRM), Generative video
- **2024:** Nobel Prize in Physics and Chemistry 2024 🏆
- **2024:** ACM Turing Award to RL 🏆



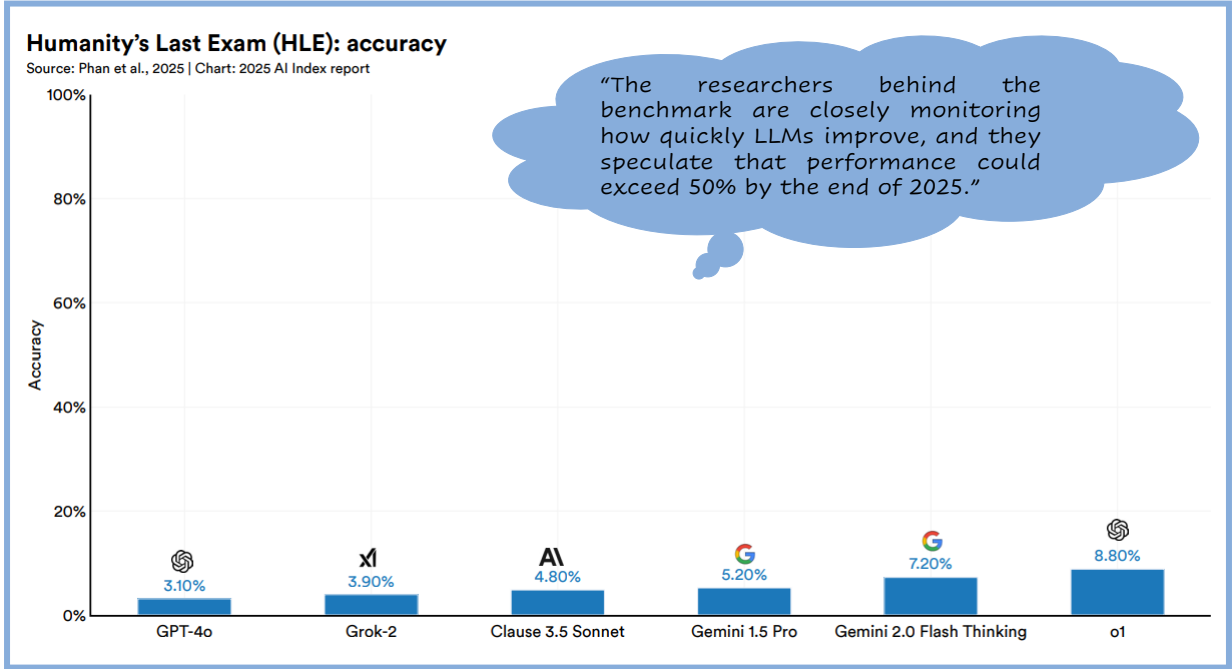
- **2025:** DeepSeek (Sputnik moment) + RL for LLM + Mixture of Experts (MoE) (January)
- **2025:** The year of agentic AI products (AI coding and more)

21



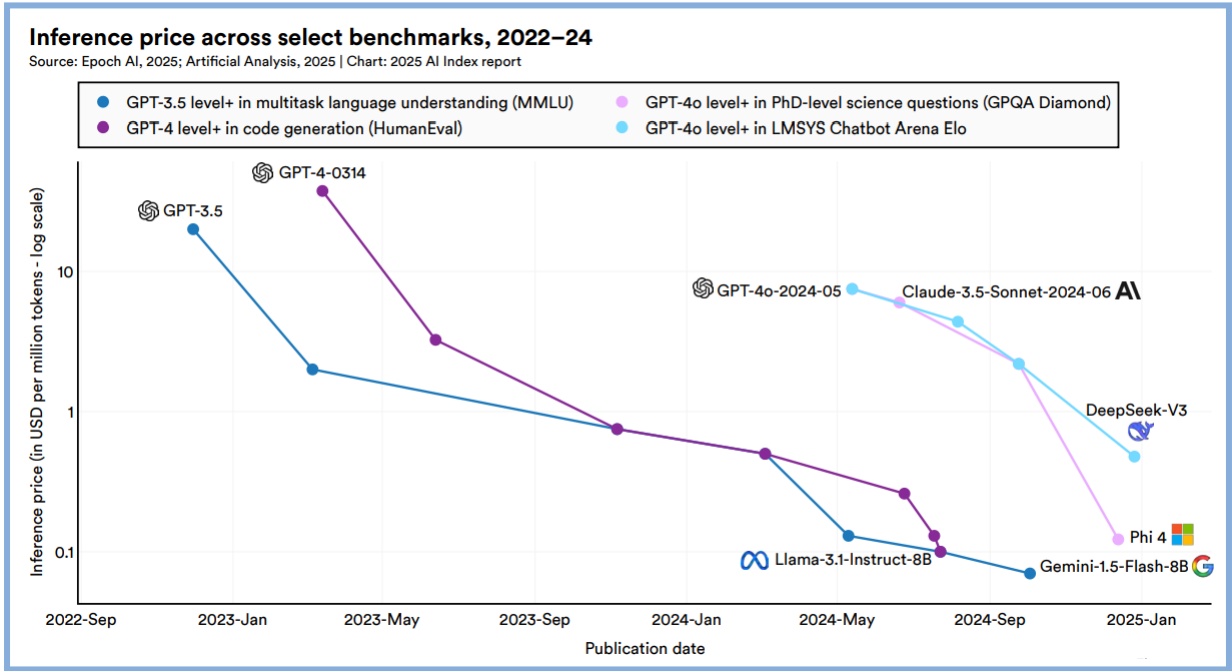
HAI AI Index Report 2025 (April 2025), https://hai.stanford.edu/assets/files/hai_ai_index_report_2025.pdf

22



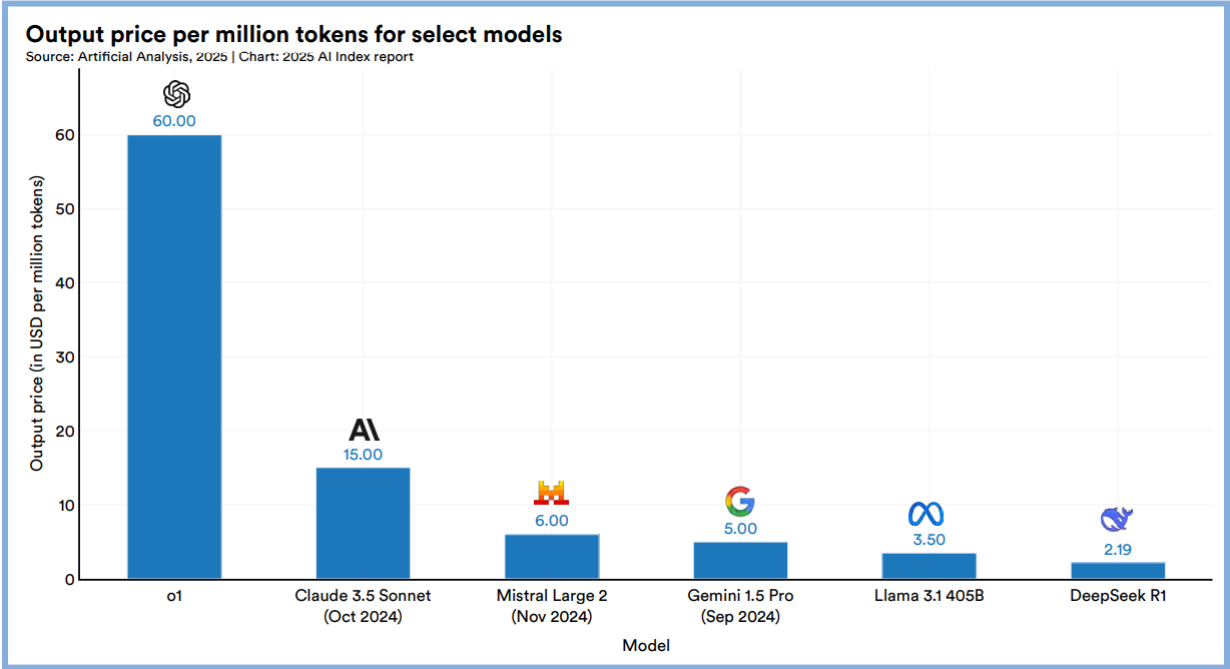
HAI AI Index Report 2025 (April 2025), https://hai.stanford.edu/assets/files/hai_ai_index_report_2025.pdf

23



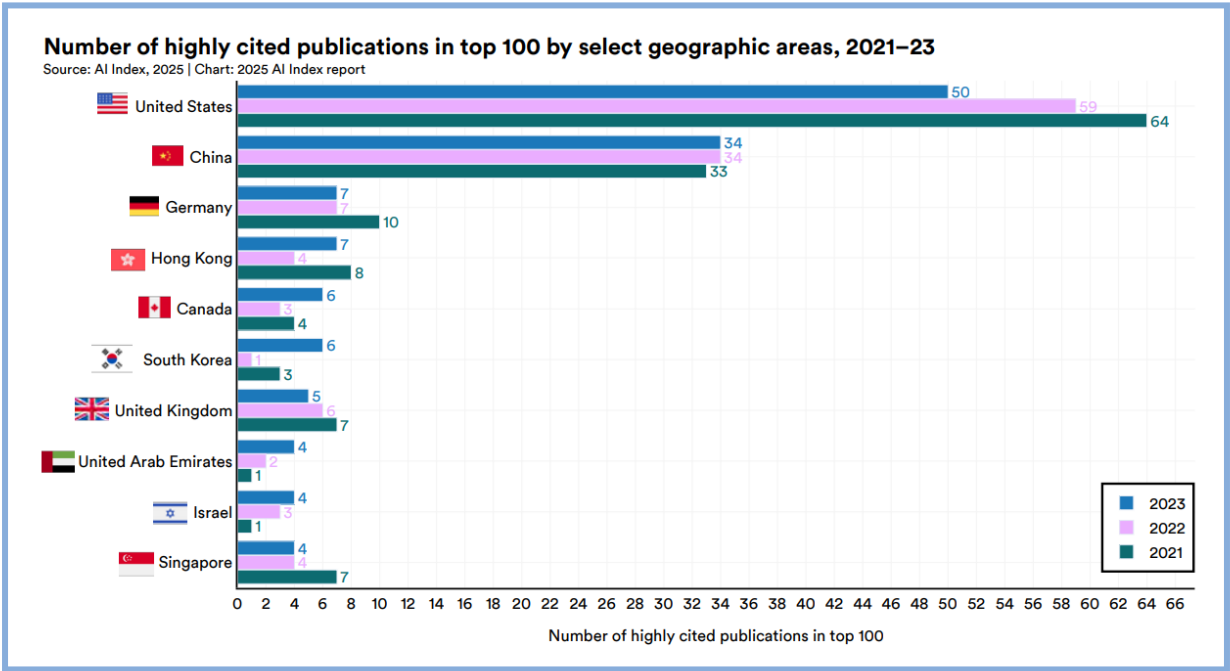
HAI AI Index Report 2025 (April 2025), https://hai.stanford.edu/assets/files/hai_ai_index_report_2025.pdf

24



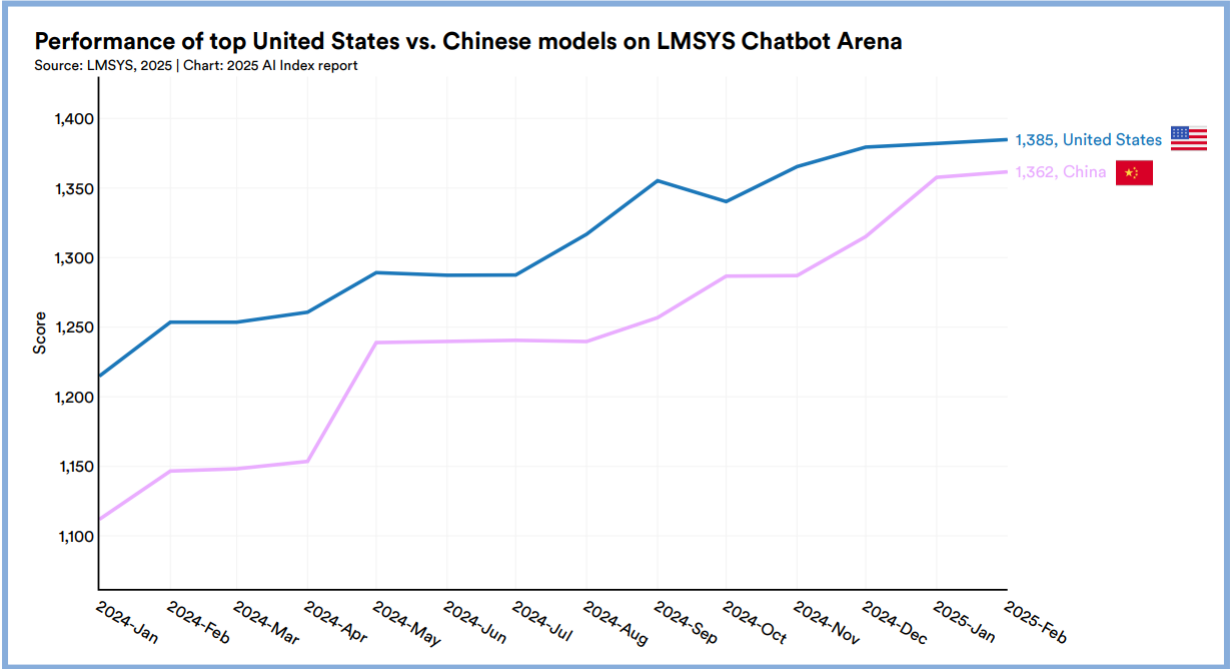
HAI AI Index Report 2025 (April 2025), https://hai.stanford.edu/assets/files/hai_ai_index_report_2025.pdf

25



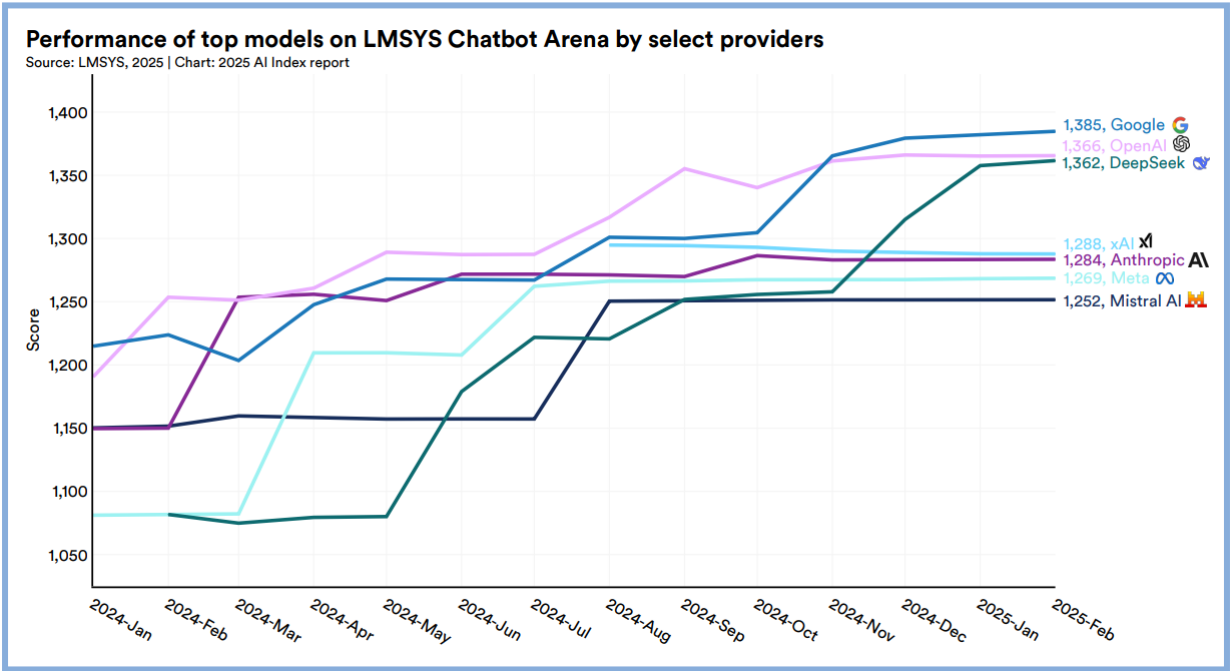
HAI AI Index Report 2025 (April 2025), https://hai.stanford.edu/assets/files/hai_ai_index_report_2025.pdf

26



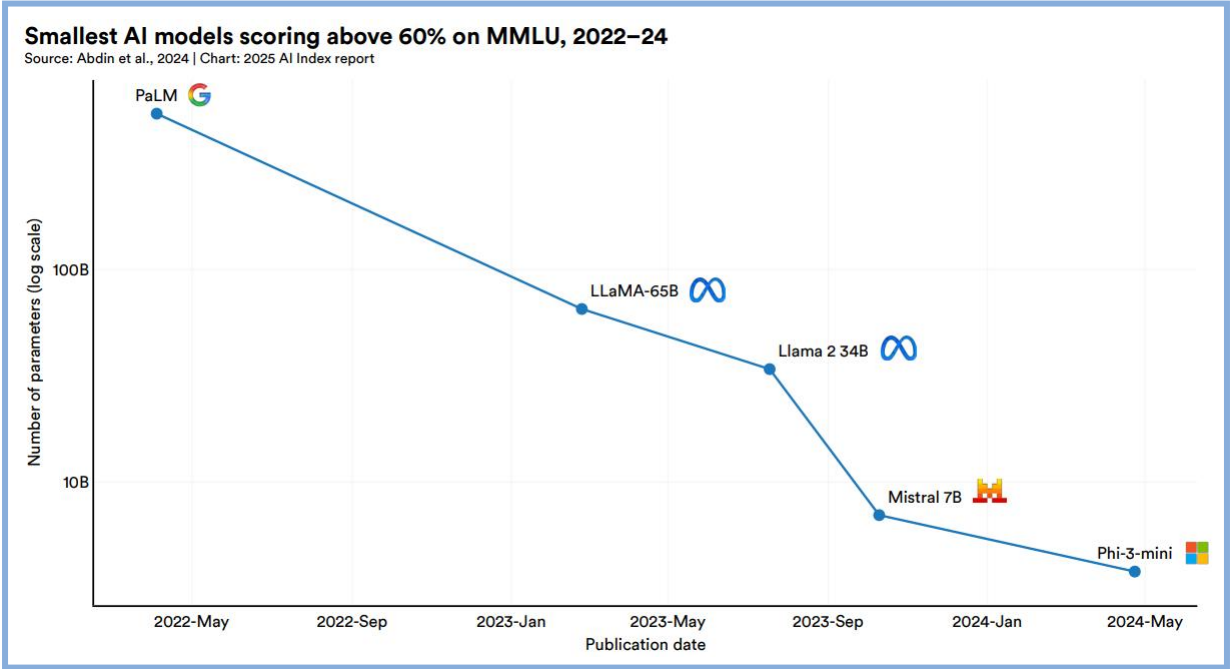
HAI AI Index Report 2025 (April 2025), https://hai.stanford.edu/assets/files/hai_ai_index_report_2025.pdf

27

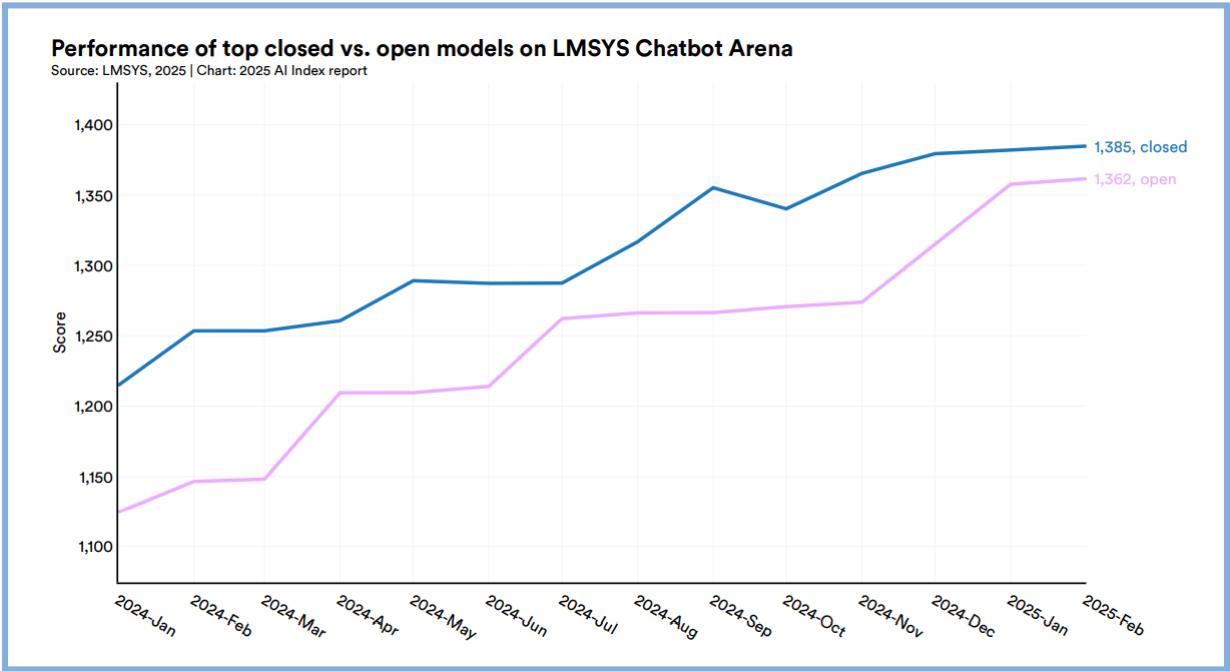


HAI AI Index Report 2025 (April 2025), https://hai.stanford.edu/assets/files/hai_ai_index_report_2025.pdf

28



29



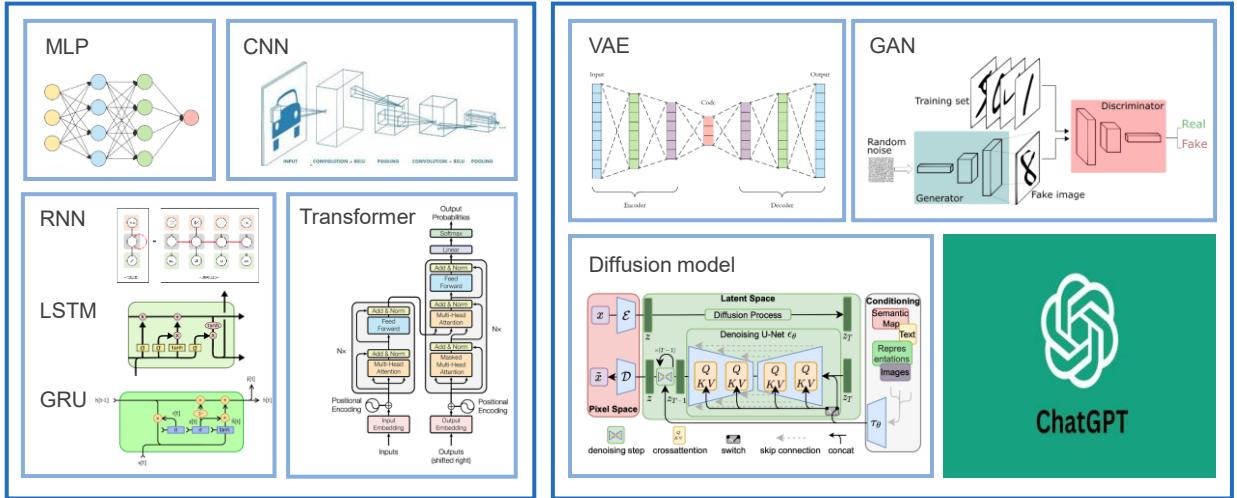
30

DL models in this course

FUNDAMENTAL

~~DISCRIMINATIVE~~

GENERATIVE



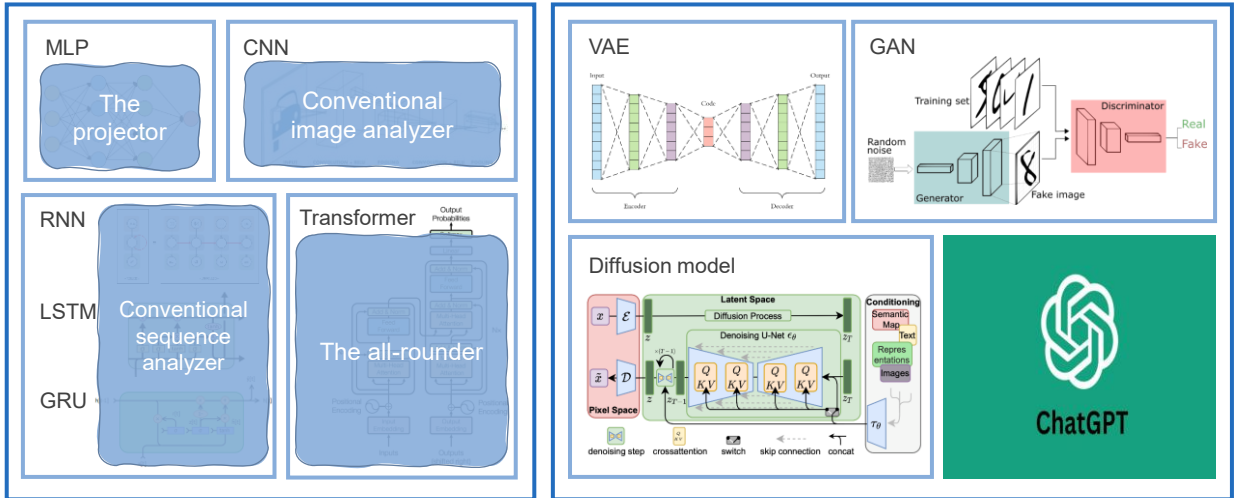
31

DL models in this course

FUNDAMENTAL

~~DISCRIMINATIVE~~

GENERATIVE

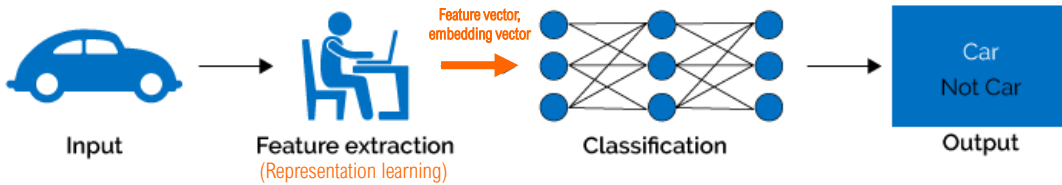


32



Paradigm shift: ML > DL

Machine Learning



Deep Learning

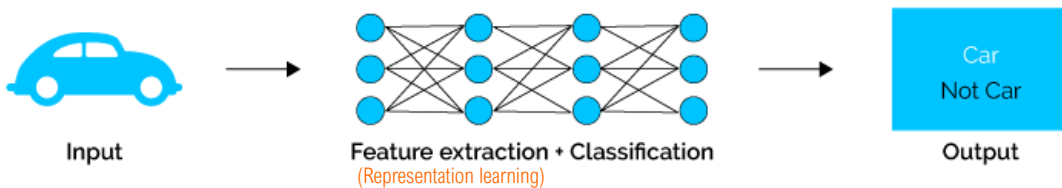
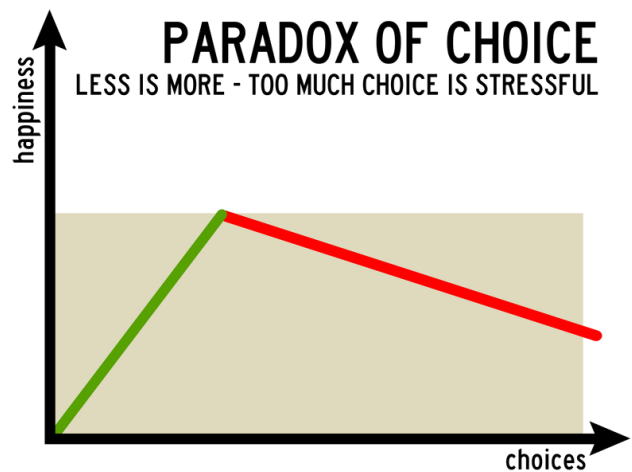
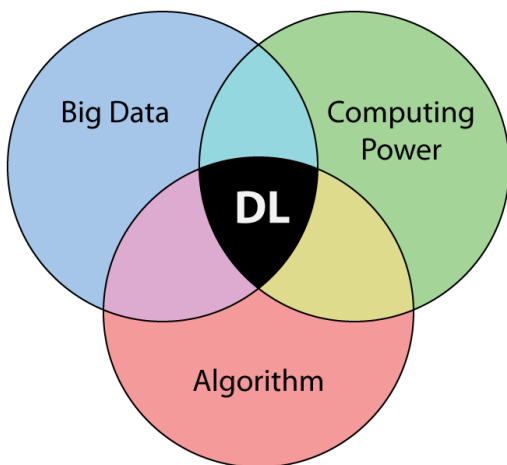


Image credit: <https://www.softwaretestinghelp.com/data-mining-vs-machine-learning-vs-ai/>

33



WHY Deep Learning



34

WHY NOT Deep Learning

- Deep learning usually requires a huge amount of input data. For supervised learning, data collection and annotation are sometimes very laborious.

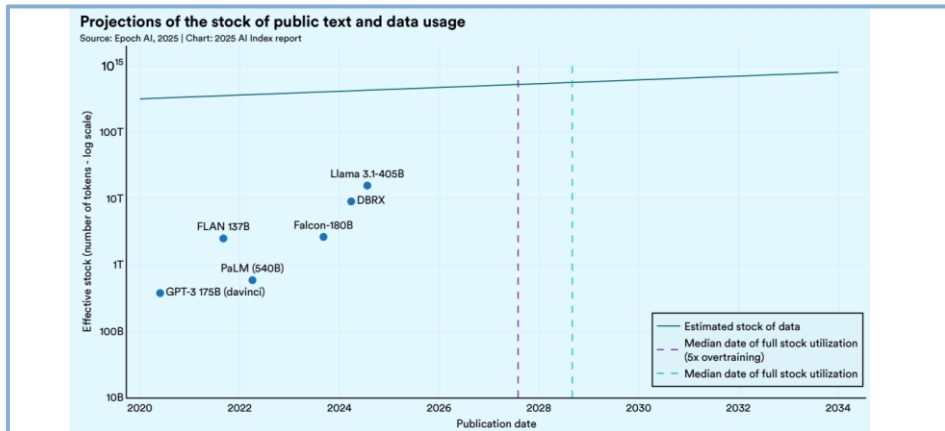


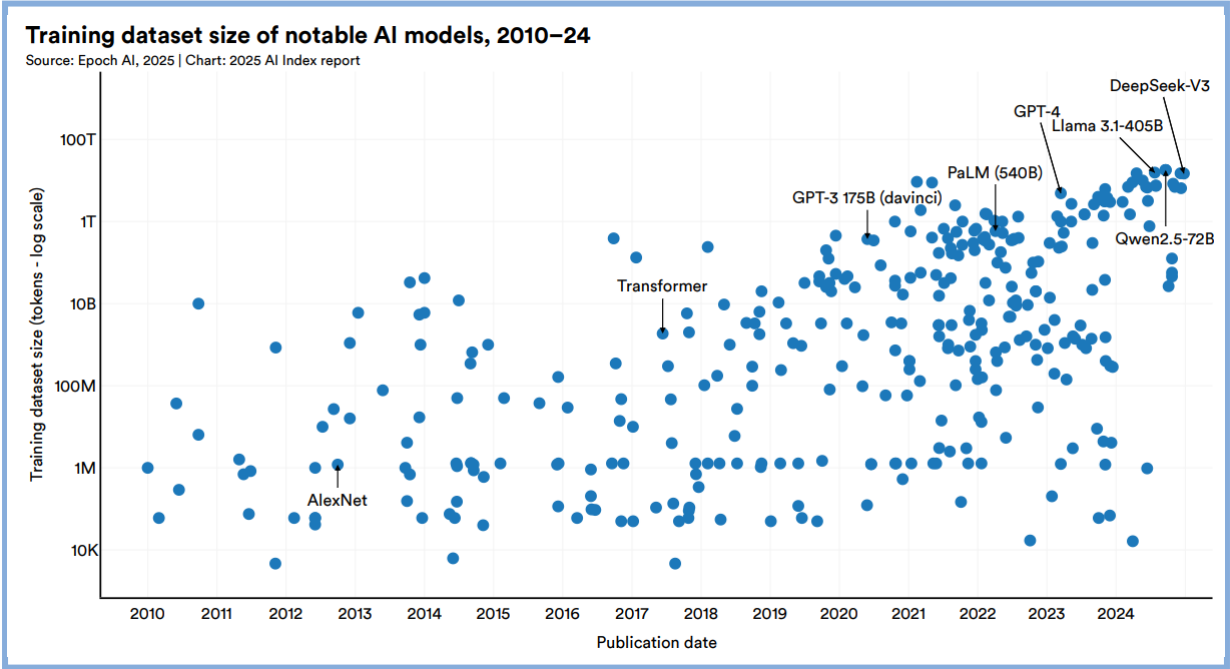
Image credit: HAI AI Index Report, April 2025, https://hai.stanford.edu/assets/files/hai_ai_index_report_2025.pdf

35

WHY NOT Deep Learning

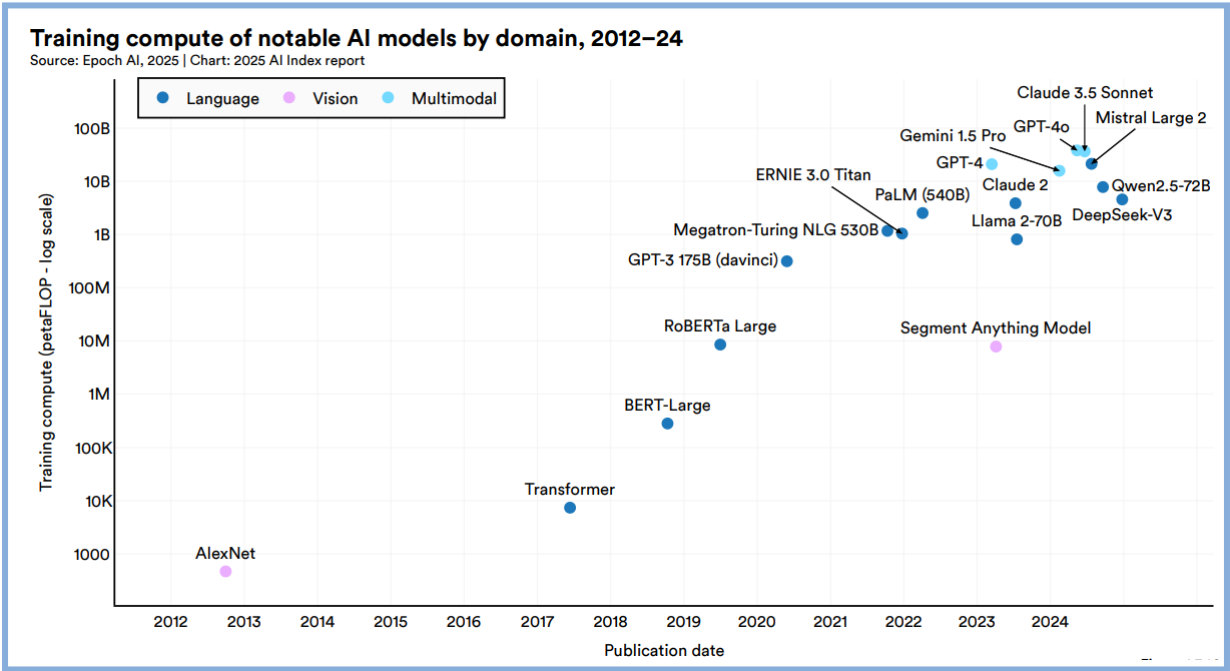
- Training deep networks consumes huge computational resources (GPU memory, training time, and inference time).
 - The largest **StyleGAN2 (Nvidia 2019)** was trained for 69d 23h on one Tesla V100 GPU.
 - At theoretical 28 TFLOPS for Tesla V100 and lowest 3-year reserved cloud pricing, **GPT-3 (OpenAI 2020)** was still trained for 355 GPU-years and cost \$4.6M.
 - **Vision Transformer (Google 2020)** was trained for 2,500 TPuv3-core-days.
 - **CLIP (OpenAI 2021)** was trained for 2 weeks on 256 GPUs.
 - **WangchanBERTa (VISTEC x DEPA 2021)** was trained for 134 days on eight Tesla V100 GPUs (one DGX-1 server).
 - **Stable Diffusion (2022)** was trained with 256 A100 GPUs on Amazon Web Services for a total of 150,000 GPU hours, at \$600,000.
 - **GPT-4 (OpenAI 2023)** used an estimated \$78 million worth of computing to train.
 - **Gemini Ultra (Google 2023)** cost \$191 million to compute.
 - In 2024, OpenAI is likely to spend inference costs around \$4 billion on processing power supplied by Microsoft (at a special discount rate), and expects to spend \$3 billion on training models and data.

36



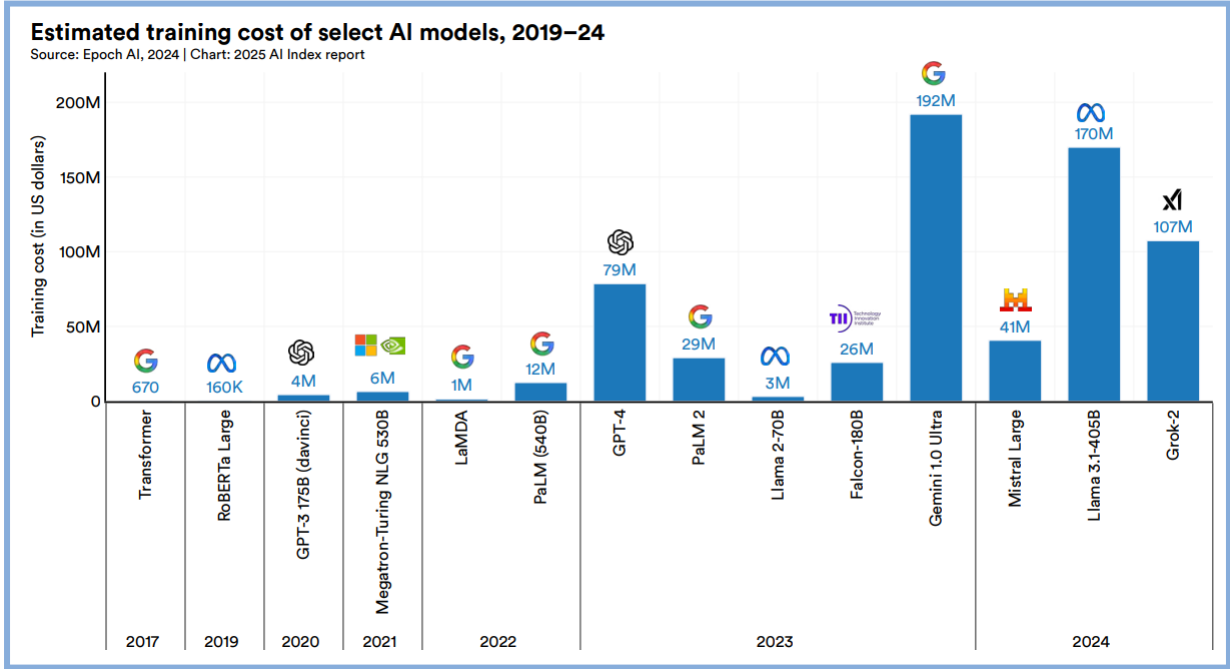
HAI AI Index Report 2025 (April 2025), https://hai.stanford.edu/assets/files/hai_ai_index_report_2025.pdf

37



HAI AI Index Report 2025 (April 2025), https://hai.stanford.edu/assets/files/hai_ai_index_report_2025.pdf

38

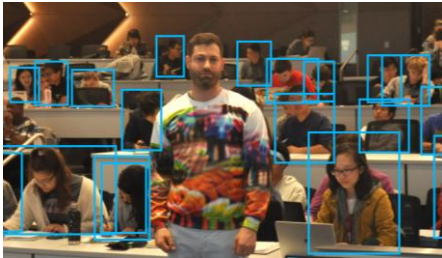
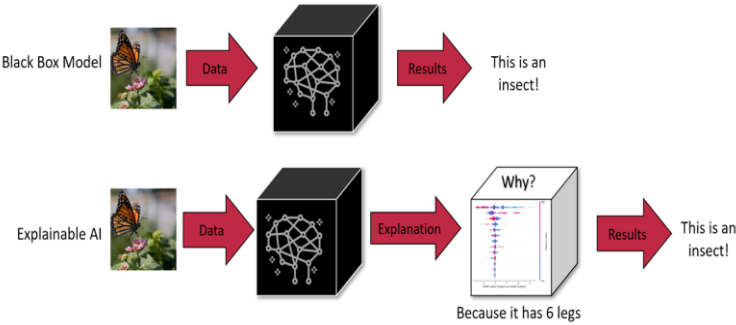


HAI AI Index Report 2025 (April 2025), https://hai.stanford.edu/assets/files/hai_ai_index_report_2025.pdf

39

WHY NOT Deep Learning

- Deep learning suffers from **the infamous black-box issue**.
 - Hence, it is necessary to use as many indicators as possible to evaluate DLs
- Deep learning can fail and can be attacked (**Adversarial Attack**).



Invisible sweater (ECCV2020):
<https://www.cs.umd.edu/~tomg/projects/invisible/>

40

A

Hardware Acceleration

Prepare necessary hardware to accelerate our deep learning project

41

REUTERS

WorldBusinessMarketsSustainabilityLegalBreakingviewsTechnology

Technology

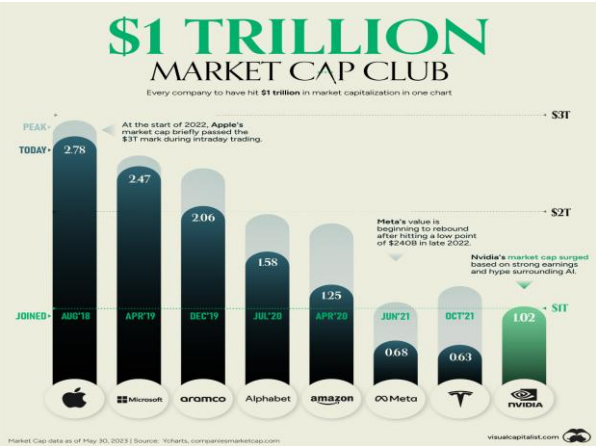
Technology

Nvidia briefly joins \$1 trillion valuation club

By Akash Sriram and Samritha A

May 31, 2023 7:33 AM GMT+7 · Updated 3 days ago

Reuters 31MAY2023 <https://www.reuters.com/technology/nvidia-sets-eye-1-trillion-market-value-2023-05-30/>



Reuters

WorldBusinessMarketsSustainabilityLegalBreakingviewsTechnologyInvestigations

Technology

Trophy

Nvidia overtakes Apple as No. 2 most valuable company

By Noel Randewich

June 6, 2024 1:38 PM GMT+7 · Updated an hour ago

Nvidia's stock market value overtakes Apple

Nvidia

Apple

Microsoft

Source: LSEG
Created by Thomson Reuters

Reuters 6JUN2024 <https://www.reuters.com/technology/nvidia-verge-overtaking-apple-no-2-most-valuable-company-2024-06-05/>

Reuters

WorldBusinessMarketsSustainabilityLegalBreakingviewsTechnology

Technology

Trophy

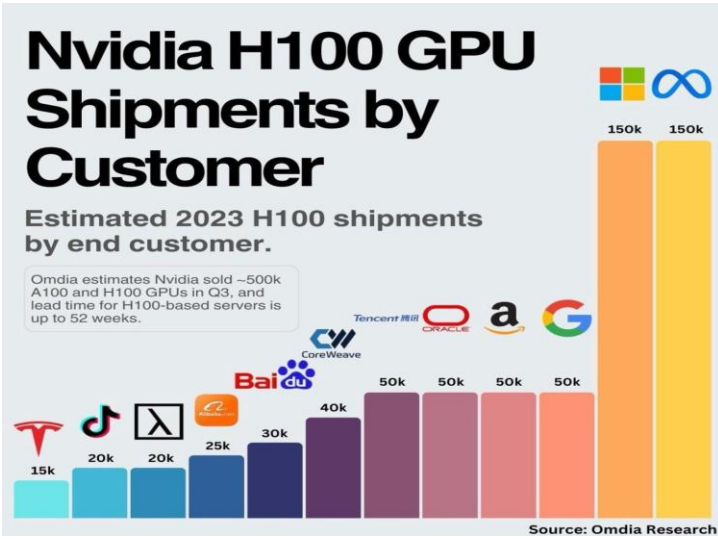
Nvidia becomes world's most valuable company

By Reuters

June 19, 2024 1:17 AM GMT+7 · Updated 2 months ago

Reuters 19JUN2024 <https://www.reuters.com/technology/view-nvidia-becomes-worlds-most-valuable-company-2024-06-18/>

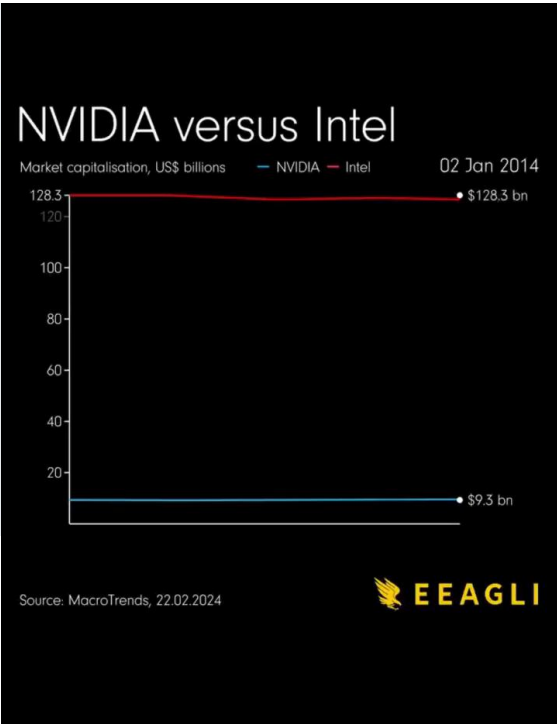
42



NVIDIA Crushes Rivals: Secures Unprecedented 90% of GPU Market in Q3 2024

Nauman khan
December 12, 2024 • 1 min read

43



Local Machine vs. Cloud

Local Machine

CPU

- Core1
- Core2
- Core3
- Core4

GPU

- Multiprocessor 1 (32 Cores)
- Multiprocessor 2 (32 Cores)
- Multiprocessor 3 (32 Cores)
- Multiprocessor 4 (32 Cores)
- Multiprocessor 13 (32 Cores)
- Multiprocessor 14 (32 Cores)

NVIDIA

RTX 2080 VS GTX 1080 Ti

Cloud

Free Google colab

Pro+ version for \$49.99/month since 8/2021
Pro version for \$9.99/month since 2/2020
Free version for public since 2018

kaggle

Free GPU since 2018 (approximately)

44

REVIEW: Google Colab



Version 1:

```
function ClickConnect(){
  console.log("Connect Clicked ~ Start");
  document.querySelector("#stop-toolbar > colab-connect-button").shadowRoot.querySelector("#connect-button").click();
  console.log("Connect Clicked ~ End");
};
setInterval(ClickConnect, 60000)
```

Version 2: If you would like to be able to stop the function, here is the new code:

```
var startClickConnect = function startClickConnect(){
  var clickConnect = function clickConnect(){
    console.log("Connect Clicked ~ Start");
    document.querySelector("#stop-toolbar > colab-connect-button").shadowRoot.querySelector("#connect-button").click();
    console.log("Connect Clicked ~ End");
  };
  var intervalId = setInterval(clickConnect, 60000);
  var stopClickConnectHandler = function stopClickConnect() {
    console.log("Connect Clicked Stopped ~ Start");
    clearInterval(intervalId);
    console.log("Connect Clicked Stopped ~ End");
  };
  return stopClickConnectHandler;
};
var stopClickConnect = startClickConnect();
```

In order to stop, call:

```
stopClickConnect();
```

Share Follow

edited Jun 7, 2020 at 17:15

Christophe Prat 17 • 5

answered Mar 7, 2020 at 15:27

barbosous 139 • 6 • 4



Credit: <https://stackoverflow.com/questions/57113226/how-can-i-prevent-google-colab-from-disconnecting>

45

REVIEW: Google Colab Pro+

Pay As You Go	Recommended Colab Pro	Colab Pro+
THB343.47 for 100 Compute Units THB1,677.76 for 500 Compute Units You currently have 0 compute units. Compute units expire after 90 days. Purchase more as you need them.	THB343.47 per month	THB1,677.76 per month
- No subscription required. Only pay for what you use. - Faster GPUs. Upgrade to more powerful premium GPUs.	100 compute units per month. Compute units expire after 90 days. Purchase more as you need them. - Faster GPUs. Upgrade to more powerful premium GPUs. - More memory. Access our higher memory machines. - Terminal. Ability to use a terminal with the connected VM.	500 compute units per month. Compute units expire after 90 days. Purchase more as you need them. - Faster GPUs. Priority access to upgrade to more powerful premium GPUs. - More memory. Access our higher memory machines. - Background execution. Upgrade your notebooks to keep executing for up to 24 hours even if you close your browser. - Terminal. Ability to use a terminal with the connected VM.

Update on AUG2023:

- Nvidia A100 GPU is available for both Colab Pro and Pro+.
- The better the GPU, the more compute units it will consume.
- When running out of compute units, we can simply buy more compute units to continue running.

• Good:

- Convenient as we can train the model while being far away from our computer or even when closing the computer
- At least 10+ hours of continuous running without interruption

• Bad:

- After 2 continuous weeks of heavy GPU usage (e.g., 4 concurrent tabs), it banned us from further GPU usage with a suggestion to move to Google Cloud ML.



Review from a real user (using Google Colab Pro+ during January-February 2022)

46

A TPU

- **TPU** is an AI accelerator hardware specially designed for neural networks and is proprietary to Google.
- **TPU** has been used to power Google's data centers internally since 2015 and in 2018 made available for 3rd party use.
- According to Google (in 2017), **TPU-v1** was about 80x faster than CPU and 30x faster than GPU at neural network inference.
- Currently, **TPU** is mostly available as a cloud service or a smaller version of the chip (Edge TPU).



TPU v1
Launched in 2015
Inference only



TPU v2
Launched in 2017
Inference and training

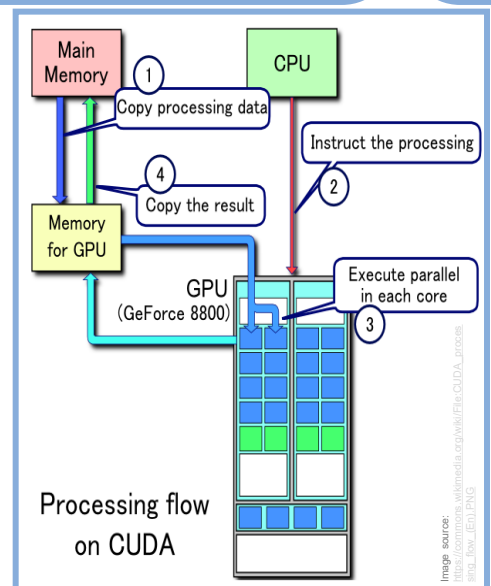
5APR2017: <https://cloud.google.com/blog/products/gcp/quantifying-the-performance-of-the-tpu-our-first-machine-learning-chip>

47

A CPU vs. GPU vs. TPU

- **Overall**, **TPU** has the highest training throughput (the number of examples trained per second).
- **Deep FC networks**:
 - For a big batch size, it is **TPU**, GPU and CPU respectively.
 - For a big model, **GPU** is the best.
 - For a big batch size and model, **GPU** is better than CPU because of better parallelization.
 - For very big models, **CPU** is better than GPU, and GPU is better than TPU. This is because CPU has the highest memory per core.
- **Deep CNNs**:
 - **TPU** is better than GPU and is the best for training CNN, particularly very big CNNs. This is because TPU is specifically optimized for spatial reuse characteristics of CNN.
- **RNNs**:
 - For speed aspect, **TPU** is a lot better than GPU.
 - Right now, TPU is good at dense computation (MatMul) whereas GPU is better in non-MatMul operations. In the future, if TPU is further optimized for non-MatMul operations, it will do even better than GPU.

[31JUL2019] Harvard University, <https://arxiv.org/abs/1907.10701>



48

AI Deep Learning's Computer



Why not Mac OS? [23NOV2019] Apple and Nvidia are over <https://gizmodo.com/apple-and-nvidia-are-over-1840015246>

How much GPU's RAM for training DL models? [18FEB2020] Choosing the Best GPU for Deep Learning in 2020 <https://lambdalabs.com/blog/choosing-a-gpu-for-deep-learning/>

Why Mac OS? [15NOV2020] How is the Apple M1 going to affect Machine Learning? <https://medium.com/disruptive-nerd/how-is-the-apple-m1-going-to-affect-machine-learning-2d9da1beef86>

49



How to choose a computer for deep learning?

50

Neural Processing Unit (NPU)



[Home](#) [AI](#) [Data Center](#) [Driving](#) [Gaming](#) [Pro Graphics](#) [Robotics](#) [Healthcare](#) [Startups](#) [AI Podcast](#) [NVIDIA Life](#)

NVIDIA RTX 500 and 1000 Professional Ada Generation Laptop GPUs Drive AI-Enhanced Workflows From Anywhere

Thin and light designs deliver advanced AI, compute and graphics horsepower for professionals on the go.

February 26, 2024 by [John Della Bona](#)

The next generation of mobile workstations with Ada Generation GPUs, including the RTX 500 and 1000 GPUs, will include both a neural processing unit (NPU), a component of the CPU, and an NVIDIA RTX GPU, which includes Tensor Cores for AI processing. The NPU helps offload light AI tasks, while the GPU provides up to an additional 682 TOPS of AI performance for more demanding day-to-day AI workflows.

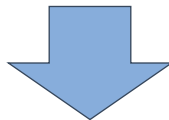
26FEB2024 <https://blogs.nvidia.com/blog/rtx-ada-ai-workflows/>

51

Neural Processing Unit (NPU)

- **Problems of GPU:**

- **Still rely on CPU:** Although GPU has the advantage in parallel computing capability, it does not work alone and needs the CPU's co-processing. The construction of neural network models and data streams are still carried out on the CPU.
- **The problem of high-power consumption and large size:** The higher the performance, the larger the GPU, the higher the power consumption, and the more expensive it is, which will not be available for some small devices and mobile devices.



- **We need a small size, low power consumption, high computational performance, and high computational efficiency of a dedicated chip. This is the birth of NPU.**

Credit: (28DEC2021) <https://www.utmel.com/blog/categories/integrated%20circuit/neural-processing-unit-npu-explained>

52

A Neural Processing Unit (NPU)

- **NPU**s are becoming increasingly common in PC, laptop, and mobile domains. It usually refers to all specialized AI processors.
 - For example: AI chip in Apple iPhone, Exynos 9 (9820) the AI-enabled NPU in Samsung mobile
- **NPU**s are designed to complement the functions of CPUs and GPUs, ensuring that no single processor gets overwhelmed, maintaining smooth operation across the system.
 - CPUs handle a broad range of tasks.
 - GPUs excel in rendering detailed graphics.
 - **NPU**s specialize in executing AI-driven tasks swiftly.

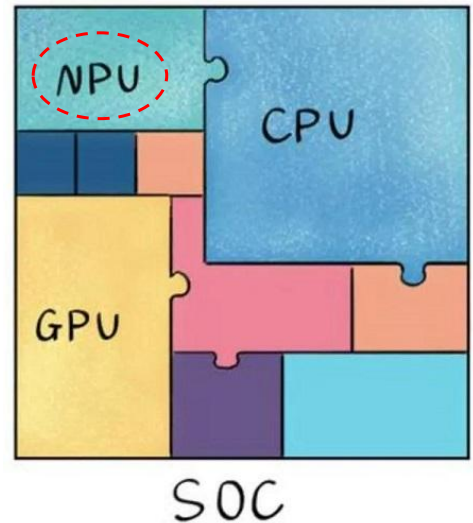


Image source: (28DEC2021) <https://www.utmel.com/blog/categories/integrated%20circuit/neural-processing-unit-npu-explained>

53

A Neural Processing Unit (NPU)

• CPU vs. GPU vs. NPU:

- Unlike traditional CPUs and GPUs, an **NPU**, at its core, is a specialized processor explicitly designed/optimized for executing AI/ML/DL algorithms, particularly for ANN mathematical computation and DL acceleration.
- This degree of specialization allows **NPU**s to deliver significantly higher performance for AI workloads compared to CPUs and even GPUs in certain scenarios.
 - **NPU**s excel in processing vast amounts of data in parallel, making them ideal for tasks like image recognition, natural language processing, and other AI-related functions.
 - For example, if you were to have an **NPU** within a GPU, the **NPU** may be responsible for a specific task like object detection or image acceleration.
- The concept of **GPNU** (GPU-NPU hybrid) has emerged, aiming to combine the strengths of GPUs and **NPU**s. GPNUs leverage the parallel processing capabilities of GPUs while integrating **NPU** architecture for accelerating AI-centric tasks.

Credit: (28DEC2021) <https://www.utmel.com/blog/categories/integrated%20circuit/neural-processing-unit-npu-explained>

54

Language Processing Unit (LPU)

-
- Output Tokens per second (tokens per second) for the 70B Models
- | Model | Output Tokens per second (tokens per second) |
|---------------|--|
| AnyScale | 66 |
| Bedrock | 21 |
| Fireworks.ai | 40 |
| Groq | 185 |
| Lepton.ai | 33 |
| Perplexity.ai | 30 |
| Replicate | 10 |
| Together.ai | 65 |

Source

- 26FEB2024 <https://www.linkedin.com/pulse/new-ai-compute-paradigm-language-processing-unit-lpu-dheeren-v%C3%A9lu-nz2bc>
- 27FEB2024 <https://www.analyticsvidhya.com/blog/2024/02/what-is-the-difference-between-lpu-and-gpu/>

Forbes

AI Supercomputer

Follow

Mar 25, 2024, 03:16pm EDT

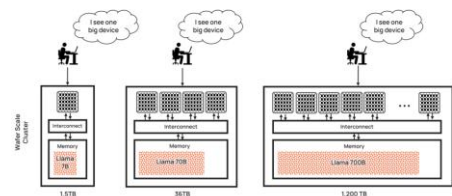


Cerebras Wafer-Scale Engine

The fastest AI chip on earth **again**

- 4 trillion transistors
- 46,226 mm² silicon
- 900,000 cores optimized for sparse linear algebra
- 5nm TSMC process
- 125 Petaflops of AI compute
- 44 Gigabytes of on-chip memory
- 21 Pbytes/s memory bandwidth
- 214 Pbit/s fabric bandwidth

- 4 trillion transistors
- 46,225 mm² silicon
- 900,000 cores optimized for sparse linear algebra
- 5nm TSMC process
- 125 Petaflops of AI compute
- 44 Gigabytes of on-chip memory
- 21 PByte/s memory bandwidth
- 214 Pbit/s fabric bandwidth



25MAR2024: <https://www.forbes.com/sites/karlfreund/2024/03/25/cerebras-update-the-wafer-scale-engine-3-is-a-door-opener/>

AI startup Cerebras debuts 'world's fastest inference' service - with a twist

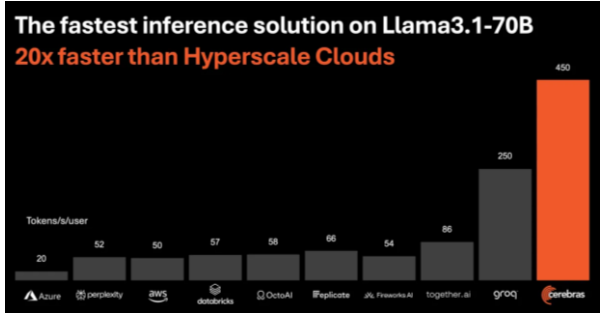
The AI computer maker claims its inference service is dramatically faster and makes new kinds of 'agentic' AI possible.



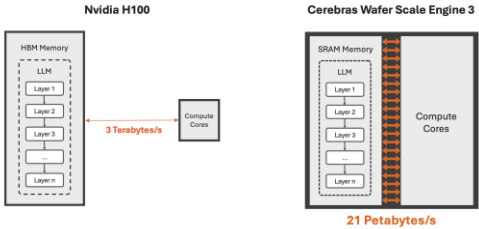
Written by Tiernan Ray, Senior Contributing Writer
Aug. 27, 2024 at 6:03 p.m. PT

Cerebras vs Nvidia H100 Llama3.1-70B

	Tokens/sec/user	Cost per M tokens ¹
Cerebras	450	\$0.60
Hyperscale H100 Cloud	20	\$2.90 ²
X-factor	22x	1/5



Storing the entire model in on-chip SRAM increases memory bandwidth by 7,000x



27AUG2024: <https://cerebras.ai/blog/introducing-cerebras-inference-ai-at-instant-speed>, <https://www.zdnet.com/article/ai-startup-cerebras-debuts-worlds-fastest-inference-with-a-twist/>

57



DL Frameworks

Which framework to use in our deep learning project

58



Image credit: <https://softwaremill.com/ml-engineer-comparison-of-pytorch-tensorflow-jax-and-flax/>

Deep Learning Frameworks

- Most deep learning frameworks can be used for two different things:
 - Replacing Numpy-like operations with GPU-accelerated operations
 - Building deep neural networks
- TensorFlow** (before 2.0) used **static computation graphs**. However, **PyTorch** uses **dynamic computation graph**, allowing greater flexibility in building complex architectures.

A graph is created on the fly

```
from torch.autograd import Variable

x = Variable(torch.randn(1, 10))
prev_h = Variable(torch.randn(1, 20))
W_h = Variable(torch.randn(20, 20))
W_x = Variable(torch.randn(20, 10))
```

W_h

h

W_x

x

59

Static vs. Dynamic Computation Graphs

- Most **static declaration paradigms** use a two-step process:
 - Define** a computational architecture that is represented as a computational graph
 - For example, take a 64x64 image and perform two layers of convolution, predict the class of the image out of 100 classes, and optionally calculate the loss with respect to the correct label
 - Execute** the computation
 - For example, a user repeatedly populates the 64x64 matrix with data and the library executes the previously declared computation graph. At test time, the predictions can be used. At training time, the loss can be calculated and back-propagated through the graph to compute the gradients required for parameter updates
- Despite a number of advantages, there are many scenarios where it is difficult for simple static declaration tools to handle.
 - For example, variably sized inputs, variably structured inputs, non-trivial inference algorithms, variably structured outputs
- In the **dynamic declaration paradigm**, a user defines the computation graph programmatically. There are no separate steps for definition and execution: the necessary computation graph is created, on the fly, as the loss calculation is executed, and a new graph is created for each training instance.
- Recommended reading materials that comparatively explain the pros and cons of each paradigm:
 - [15JAN2017] DyNet: The Dynamic Neural Network Toolkit <https://arxiv.org/abs/1701.03980>
 - [11MAR2019] <https://medium.com/analytics-vidhya/dynamic-vs-static-computation-graph-2579d1934ecf>

60

Deep Learning Frameworks



since 2015 (with several backends including TF)
Since 6/2019, officially included in TF 2.0
since 2023, Keras 3.0 includes full APIs for TF, JAX, and Torch backends



since October 2016



since February 2024 (stable release)



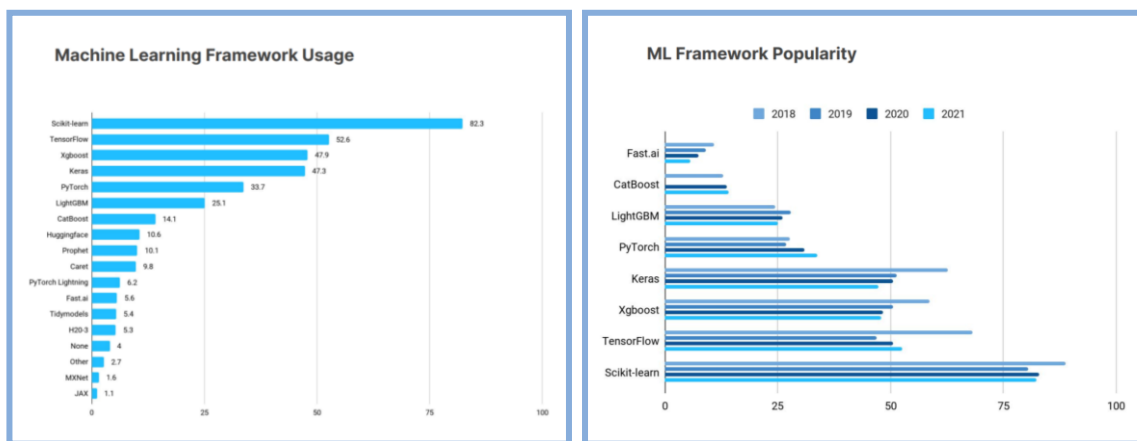
since 2015
since 6/2019 (TF 2.0), include keras's high-level APIs

Simply change from:
`from keras.XXX import YYY`
To:
`from tensorflow.keras.XXX import YYY`

61

Google TensorFlow vs. Meta PyTorch

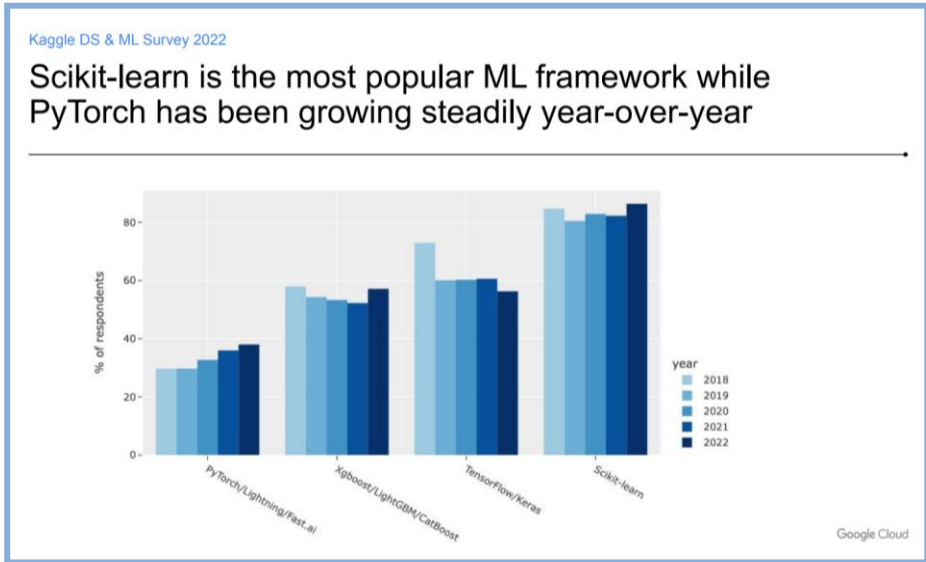
- Previously, TensorFlow took the lead when considering the number of users. It has been a go-to framework for deployment-oriented applications.



[14OCT2021] Kaggle: <https://www.kaggle.com/kaggle-survey-2021>

62

Google TensorFlow vs. Meta PyTorch



[11-13OCT2022] Kaggle:
<https://www.kaggle.com/kaggle-survey-2022>

63

Google TensorFlow vs. Meta PyTorch

- PyTorch quickly became the preferred choice within research/developer communities.

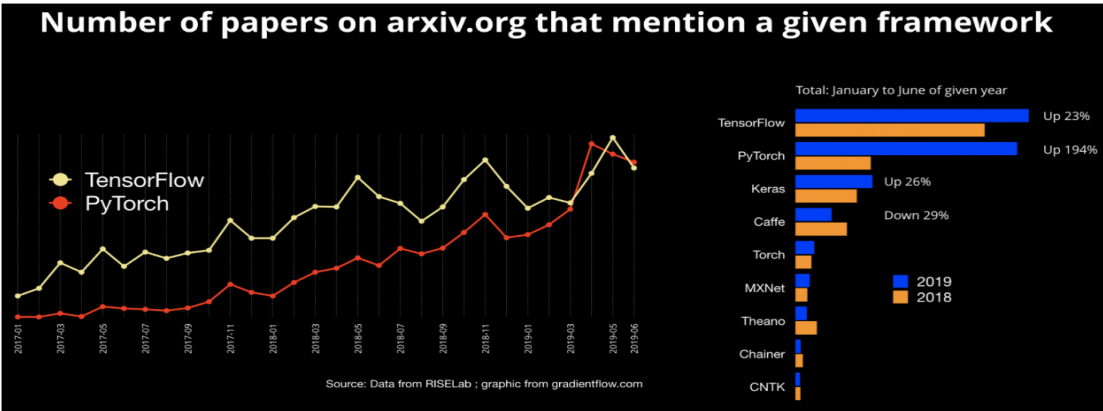


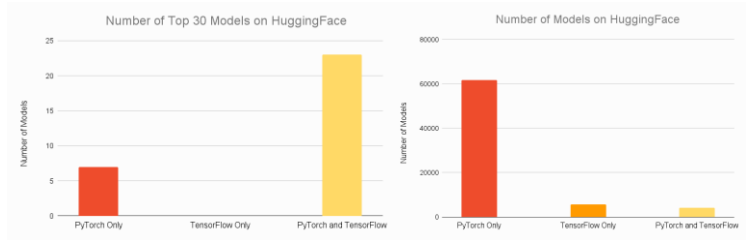
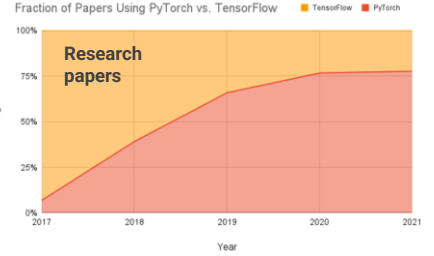
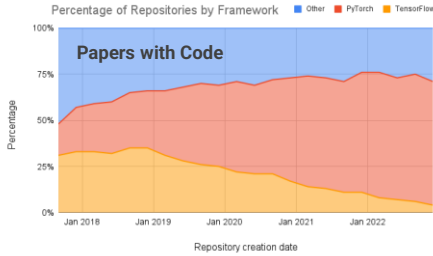
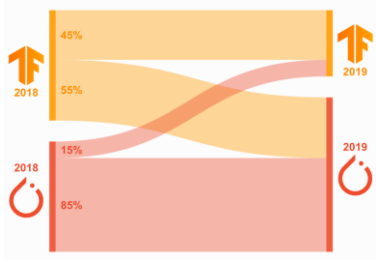
Figure 1. Number of papers posted on arXiv.org that mention each framework. Source: Data from RISELab and graphic by Ben Lorica.

[JULY2019] <https://www.oreilly.com/content/one-simple-graphic-researchers-love-pytorch-and-tensorflow/>

64

Google TensorFlow vs. Meta PyTorch

- PyTorch has steadily overshadowed TensorFlow.

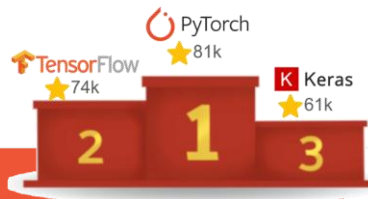
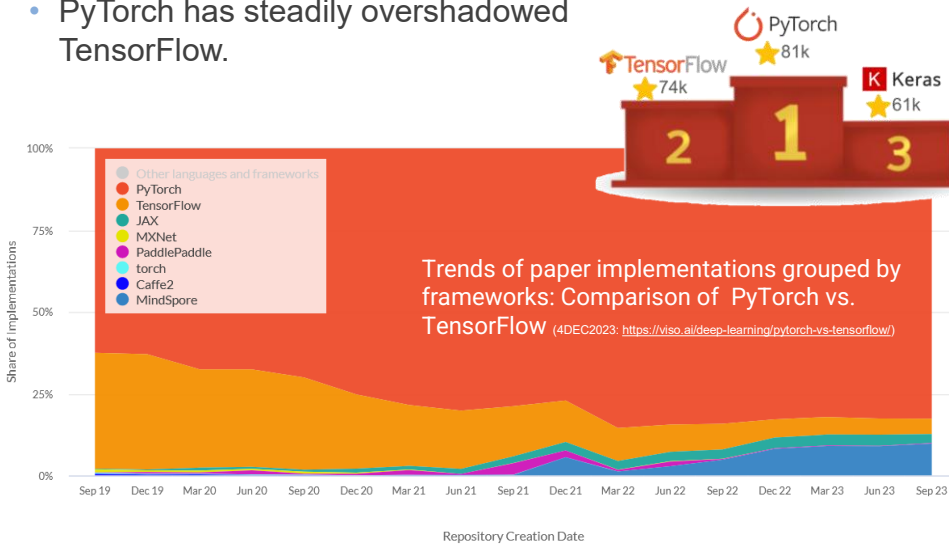


14DEC2021: <https://www.assemblyai.com/blog/pytorch-vs-tensorflow-in-2023/>

65

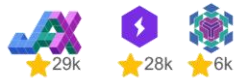
Google TensorFlow vs. Meta PyTorch

- PyTorch has steadily overshadowed TensorFlow.



Popularity on Github

24FEB2024: <https://softwaremill.com/ml-engineer-comparison-of-pytorch-tensorflow-jax-and-flax/>



66

Deep Learning Frameworks



since 2015 (with several backends including TF)
Since 6/2019, officially included in TF 2.0
since 2023, Keras 3.0 includes full APIs for TF, JAX, and Torch backends



since October 2016



since February 2024 (stable release)



since 2015
since 6/2019 (TF 2.0), include keras's high-level APIs

Simply change from:
from keras.XXX import YYY
To:
from tensorflow.keras.XXX import YYY

67

```
import keras

model = keras.Sequential([
    keras.layers.Input(shape=(num_features,)),
    keras.layers.Dense(512, activation="relu"),
    keras.layers.Dense(512, activation="relu"),
    keras.layers.Dense(num_classes, activation="softmax"),
])
model.summary()

model.compile(
    optimizer=keras.optimizers.AdamW(learning_rate=1e-3),
    loss=keras.losses.CategoricalCrossentropy(),
    metrics=[
        keras.metrics.CategoricalAccuracy(),
        keras.metrics.AUC(),
    ],
)

history = model.fit(
    x_train, y_train, batch_size=64, epochs=8, validation_split=0.2
)
evaluation_scores = model.evaluate(x_val, y_val, return_dict=True)
predictions = model.predict(x_test)
```



\$ python example.py
Using TensorFlow backend



\$ python example.py
Using PyTorch backend



\$ python example.py
Using JAX backend



Image credit: "Introducing Keras 3.0," https://keras.io/keras_3/

68

Develop
custom components
that work with
any framework
using `keras.ops`
(which includes the
NumPy API)

```
import keras
from keras import ops

class TokenEmbedPositionEmbedding(keras.Layer):
    def __init__(self, num_tokens, vocab_size, embed_dim):
        super().__init__()
        self.token_embed = self.add_weight(
            shape=(vocab_size, embed_dim),
            initializer="random_uniform",
            trainable=True,
        )
        self.position_embed = self.add_weight(
            shape=(num_tokens, embed_dim),
            initializer="random_uniform",
            trainable=True,
        )

    def call(self, token_ids):
        lengths = token_ids.shape[-1]
        positions = ops.arange(0, lengths, dtype="int32")
        position_vectors = ops.take(self.position_embed, positions, axis=0)
        token_ids = ops.cast(token_ids, dtype="int32")
        token_vectors = ops.take(self.token_embed, token_ids, axis=0)
        # Sum each
        embed = token_vectors + position_vectors
        # Normalize embeddings
        power_sum = ops.sum(ops.square(embed), axis=-1, keepdims=True)
        return embed / ops.sqrt(ops.maximum(power_sum, ops.convert_to_numpy(1e-7)))
```

```
import torch
class TokenEmbedPositionEmbedding(keras.Layer):
    def __init__(self, num_tokens, vocab_size, embed_dim):
        super().__init__()
        self.token_embed = self.add_weight(
            shape=(vocab_size, embed_dim),
            initializer="random_uniform",
            trainable=True,
        )
        self.position_embed = self.add_weight(
            shape=(num_tokens, embed_dim),
            initializer="random_uniform",
            trainable=True,
        )

    def call(self, token_ids):
        lengths = token_ids.shape[-1]
        positions = ops.arange(0, lengths, dtype="int32")
        position_vectors = ops.take(self.position_embed, positions, axis=0)
        token_ids = ops.cast(token_ids, dtype="int32")
        token_vectors = ops.take(self.token_embed, token_ids, axis=0)
        # Sum each
        embed = token_vectors + position_vectors
        # Normalize embeddings
        power_sum = ops.sum(ops.square(embed), axis=-1, keepdims=True)
        return embed / ops.sqrt(ops.maximum(power_sum, ops.convert_to_numpy(1e-7)))
```

...
or use
your framework of choice
for backend-specific
components

```
import torch
class TokenEmbedPositionEmbedding(keras.Layer):
    def __init__(self, num_tokens, vocab_size, embed_dim):
        super().__init__()
        self.token_embed = self.add_weight(
            shape=(vocab_size, embed_dim),
            initializer="random_uniform",
            trainable=True,
        )
        self.position_embed = self.add_weight(
            shape=(num_tokens, embed_dim),
            initializer="random_uniform",
            trainable=True,
        )

    def call(self, token_ids):
        lengths = token_ids.shape[-1]
        positions = ops.arange(0, lengths, dtype="int32")
        position_vectors = ops.take(self.position_embed, positions, axis=0)
        token_ids = ops.cast(token_ids, dtype="int32")
        token_vectors = ops.take(self.token_embed, token_ids, axis=0)
        # Sum each
        embed = token_vectors + position_vectors
        # Normalize embeddings
        power_sum = ops.sum(ops.square(embed), axis=-1, keepdims=True)
        return embed / ops.sqrt(ops.maximum(power_sum, ops.convert_to_numpy(1e-7)))
```

```
import tensorflow as tf
class TokenEmbedPositionEmbedding(keras.Layer):
    def __init__(self, num_tokens, vocab_size, embed_dim):
        super().__init__()
        self.token_embed = self.add_weight(
            shape=(vocab_size, embed_dim),
            initializer="random_uniform",
            trainable=True,
        )
        self.position_embed = self.add_weight(
            shape=(num_tokens, embed_dim),
            initializer="random_uniform",
            trainable=True,
        )

    def call(self, token_ids):
        lengths = token_ids.shape[-1]
        positions = ops.arange(0, lengths, dtype="int32")
        position_vectors = ops.take(self.position_embed, positions, axis=0)
        token_ids = ops.cast(token_ids, dtype="int32")
        token_vectors = ops.take(self.token_embed, token_ids, axis=0)
        # Sum each
        embed = token_vectors + position_vectors
        # Normalize embeddings
        power_sum = ops.sum(ops.square(embed), axis=-1, keepdims=True)
        return embed / ops.sqrt(ops.maximum(power_sum, ops.convert_to_numpy(1e-7)))
```

```
model = get_keras_core_model()
optimizer = keras.optimizers.Adam(learning_rate=3e-3)
loss_fn = torch.nn.CrossEntropyLoss()

def train_step(inputs, targets):
    # Compute loss
    logits = model(inputs, training=True)
    loss = loss_fn(logits, targets)

    # Compute gradients
    model.zero_grad()
    loss.backward()

    # Update weights
    optimizer.step()
    return loss

# Iterate over epochs
for epoch in range(num_epochs):
    # Iterate over the batches of the dataset
    for step, (inputs, targets) in enumerate(dataset):
        loss = train_step(inputs, targets)
        print(f'Loss: {loss.detach().numpy():.4f}')
```

```
model = get_keras_core_model()
optimizer = keras.optimizers.Adam(learning_rate=3e-3)
loss_fn = keras.losses.CategoricalCrossentropy(from_logits=True)

def function jit_compile=True):
    def train_step(inputs, targets):
        # Compute loss
        with tf.GradientTape() as tape:
            logits = model(inputs, training=True)
            loss = loss_fn(targets, logits)

        # Compute gradients
        gradients = tape.gradient(loss, model.trainable_weights)
        optimizer.apply_gradients(optimizer.get_variables())
        return loss

    # Iterate over epochs
    for epoch in range(num_epochs):
        # Iterate over the batches of the dataset
        for step, (inputs, targets) in enumerate(dataset):
            loss = train_step(inputs, targets)
            print(f'Loss: {loss.numpy():.4f}')
```

Writing a custom
training loop for
a Keras model:
PyTorch,
TensorFlow,
JAX

```
model = get_keras_core_model()
optimizer = keras.optimizers.Adam(learning_rate=3e-3)
loss_fn = keras.losses.CategoricalCrossentropy(from_logits=True)

# All variables must be built before training starts
optimizer.build(model.trainable_variables)

def compute_loss_and_updates(trainable_vars, non_trainable_vars, data):
    # Distinct function to compute the loss and non-trainable var updates
    x, y = data
    pred, non_trainable_vars = model.stateless_call(trainable_vars, non_trainable_vars, x)
    loss = loss_fn(pred, y)
    return loss, non_trainable_vars

# Function that returns the gradients for the trainable vars
grad_fn = jax.value_and_grad(compute_loss_and_updates, has_aux=True)

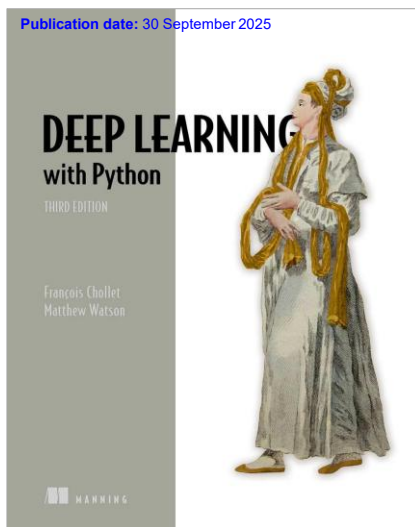
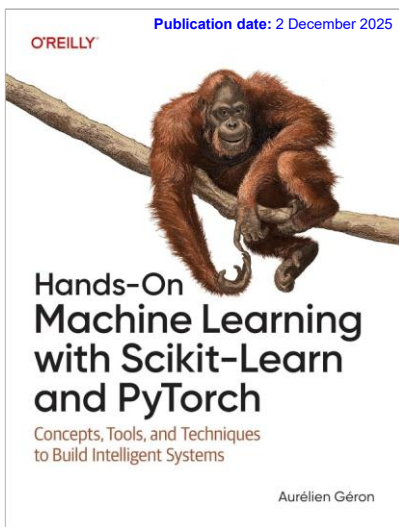
@jit
def train_step(state, data):
    # Distinct function that calls the grad fn and computes trainable vars updates
    trainable_vars, non_trainable_vars, optimizer_vars = state
    (loss, non_trainable_vars), grads = grad_fn(trainable_vars, non_trainable_vars, data)
    trainable_vars, optimizer_vars = optimizer.stateless_apply(
        optimizer_vars, grads, trainable_vars
    )
    return loss, (trainable_vars, non_trainable_vars, optimizer_vars)

# Prepare model, train state
state = (model.trainable_variables, model.non_trainable_variables, optimizer_variables)

# Iterate over epochs
for epoch in range(num_epochs):
    # Iterate over the batches of the dataset
    for step, (inputs, targets) in enumerate(dataset):
        # Use train_step call in entirely oblivious (no side effects).
        loss, state = train_step(state, data)
        print(f'Loss: {loss:.4f}')
```

Image credit: "Introducing Keras 3.0," https://keras.io/keras_3/

Google TensorFlow vs. Meta PyTorch





DL frameworks in this course



python™



Keras

PYTORCH

71



Keras Ecosystem

- **KerasTuner (2019): Hyperparam Tuning** https://keras.io/keras_tuner/
 - An easy-to-use, scalable hyperparameter optimization framework
- **KerasCV (2022)**
- **KerasNLP (2022)**
- **KerasHub (2024): Multi-framework Pretrained Models** <https://github.com/keras-team/keras-hub>
 - Implementations of popular model architectures, paired with a collection of pretrained checkpoints available on Kaggle Models
- **AutoKeras (2020): An AutoML system based on Keras** <https://autokeras.com/>

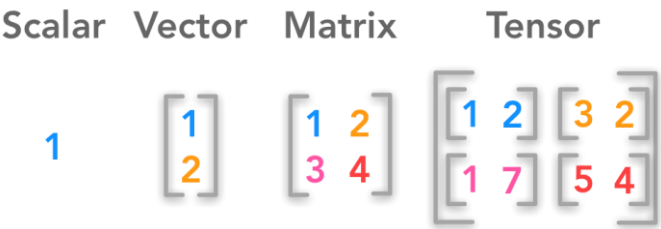
72



73

AI Tensor in Deep Learning

- **Tensor** can be thought as a general term for n d-array where n can be 0, 1, 2, 3,



How many indexes required?	Computer Science	Mathematics	
0	Number	Scalar	Rank-0 tensor (0 dimension)
1	Array	Vector	Rank-1 tensor (1 dimension)
2	2D-array	Matrix	Rank-2 tensor (2 dimensions)
n	nd-array	nd-tensor	Rank-n tensor (n dimensions)

74

A FLOPS vs. FLOPs

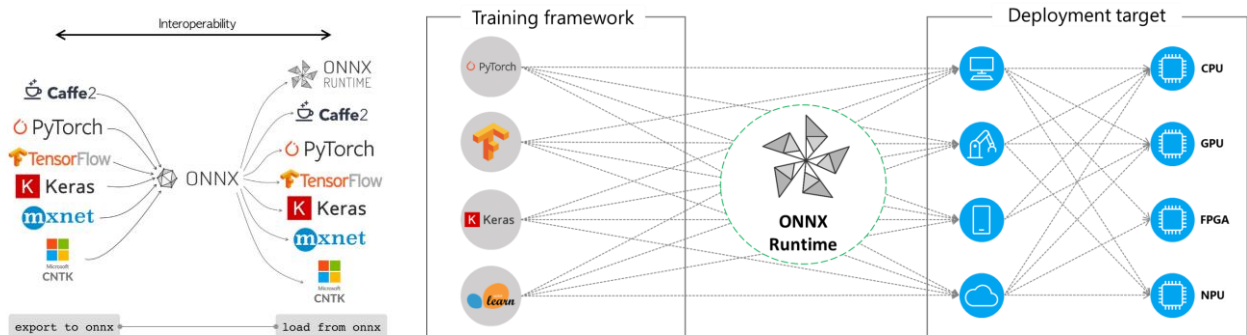
- One **floating-point operation** refers to one multiplication, one division, one addition, and so on.
- **FLOPS** (**F**loating-point **O**perations per **S**econd) is a unit of speed that often uses to measure computing hardware performance.
 - For example, NVIDIA RTX 2080 has a theoretical performance in FP32 mode of 10.07 TFLOPS. This means the theoretical maximum number of floating-point operations that the hardware might be capable of (if we are extremely lucky).
 - For modern CPUs, **FLOPS** is calculated from repeated uses of a “fused multiply then add” instruction, so that one instruction counts as two floating point operations.
- **FLOPs** (**F**loating-point **O**perations) is a unit of amount that often uses to describe how many operations are required to run a single instance of a given model.

Model	Input Size	Param Size	Flops
AlexNet	227 x 227	233 MB	727 MFLOPs
CaffeNet	224 x 224	233 MB	724 MFLOPs
VGG-VD-16	224 x 224	528 MB	16 GFLOPs
VGG-VD-19	224 x 224	548 MB	20 GFLOPs
GoogLeNet	224 x 224	51 MB	2 GFLOPs
ResNet-34	224 x 224	83 MB	4 GFLOPs
ResNet-152	224 x 224	230 MB	11 GFLOPs
SENet	224 x 224	440 MB	21 GFLOPs

75

A ONNX , ONNX Runtime

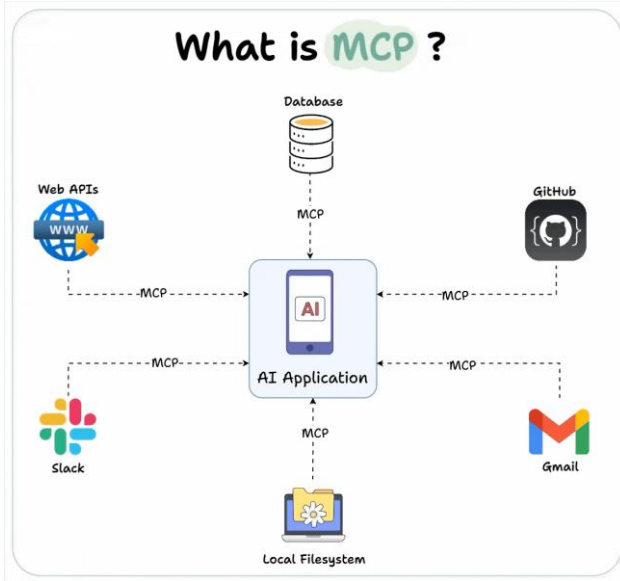
- Open Neural Network Exchange (**ONNX**)



Images from: <https://www.aurigait.com/blog/onnx-onnx-runtime-and-tensorrt/> , <https://onnxruntime.ai/docs/execution-providers/>

76

Model Context Protocol



- **MCP** is like a universal connector for AI applications, allowing different AI tools and models to interact seamlessly with various data sources.
- **MCP** is an open protocol that standardizes how applications provide context to LLMs.
- **MCP** was originally built to improve Claude's ability to interact with external systems. Anthropic decided to open-source MCP in early 2024 to encourage industry-wide adoption.
- Read more <https://www.ibm.com/think/topics/model-context-protocol>

Image credit: (29JUL2025) <https://abhishek-iiit.medium.com/model-context-protocol-mcp-assets-8cae94599875>

77

Carbon Emissions Problem

Common carbon footprint benchmarks

in lbs of CO₂ equivalent

Roundtrip flight b/w NY and SF (1 passenger)	1,984
Human life (avg. 1 year)	11,023
American life (avg. 1 year)	36,156
US car including fuel (avg. 1 lifetime)	126,000
Transformer (213M parameters) w/ neural architecture search	626,155

Chart: MIT Technology Review • Source: Strubell et al. • Created with Datawrapper



	Date of original paper	Energy consumption (kWh)	Carbon footprint (lbs of CO ₂ e)	Cloud compute cost (USD)
Transformer (65M parameters)	Jun, 2017	27	26	\$41-\$140
Transformer (213M parameters)	Jun, 2017	201	192	\$289-\$981
ELMo	Feb, 2018	275	262	\$433-\$1,472
BERT (110M parameters)	Oct, 2018	1,507	1,438	\$3,751-\$12,571
Transformer (213M parameters) w/ neural architecture search	Jan, 2019	656,347	626,155	\$942,973-\$3,201,722
GPT-2	Feb, 2019	-	-	\$12,902-\$43,008

Note: Because of a lack of power draw data on GPT-2's training hardware, the researchers weren't able to calculate its carbon footprint.
Table: MIT Technology Review • Source: Strubell et al. • Created with Datawrapper



(6JUN2019) MIT Technology Review: <https://www.technologyreview.com/2019/06/06/239031/training-a-single-ai-model-can-emit-as-much-carbon-as-five-cars-in-their-lifetimes/>

78

A Carbon Emissions Problem

- In the DL era, the computational resources needed to produce a best-in-class AI model has on average doubled every 3.4 months; this translates to a 300,000x increase between 2012 and 2018.
- A study in 2019 estimated that:
 - Training a single DL model (a particularly energy-intensive model) can generate up to 626,155 pounds of CO₂ emissions—roughly equal to the total lifetime carbon footprint of five cars. As a point of comparison, the average American generates 36,156 pounds of CO₂ emission in a year.
 - Training an average-sized model (much smaller than GPT) and examining not just the energy required to train the final version, but the total number of trial runs that went into producing the final version:
 - Over the course of six months, 4,789 different versions of the model were trained, requiring 9,998 total days' worth of GPU time (more than 27 years).
 - Taking all these runs into account, the researchers estimated that building this model generated over 78,000 pounds of CO₂ emission in total—more than the average American adult will produce in two years.

(17JUN2020): <https://www.forbes.com/sites/robtoews/2020/06/17/deep-learning-climate-change-problem/?sh=7120e1176b43>

79

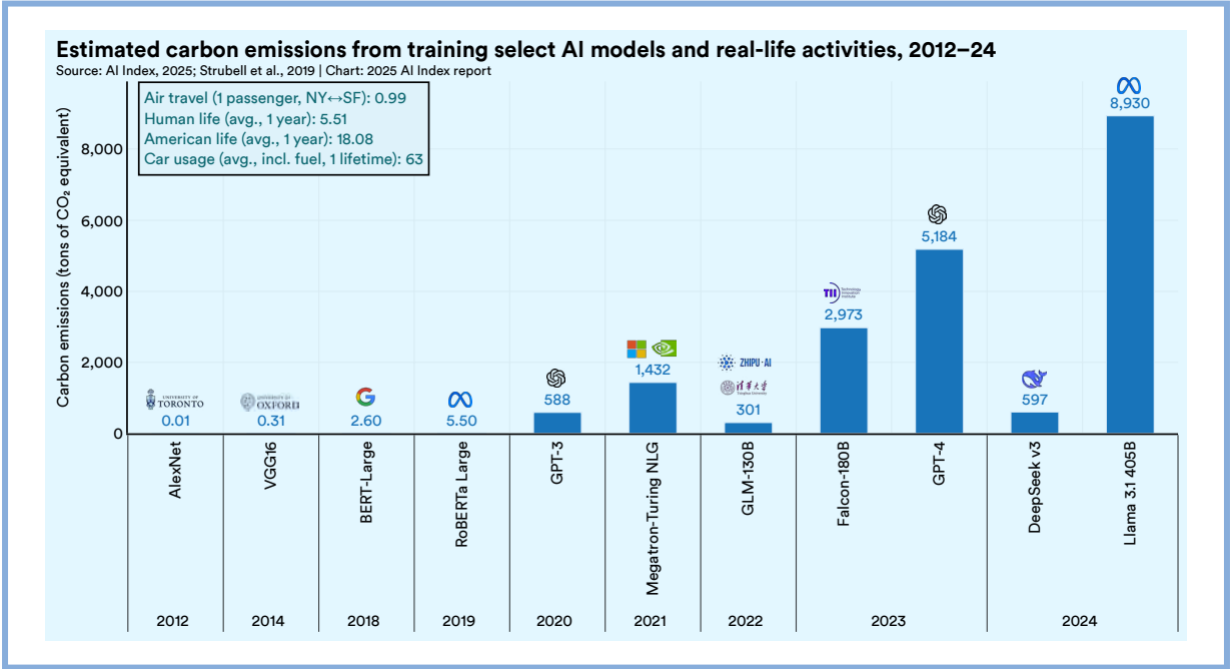
A Carbon Emissions Problem

- Deploying AI models to take action in real-world settings—a process known as inference—comes even more energy than training does. NVIDIA estimates that 80-90% of the cost of a neural network is in inference rather than training.
- Machine Learning Emissions Calculator <https://mlco2.github.io/impact/#compute>



(17JUN2020): <https://www.forbes.com/sites/robtoews/2020/06/17/deep-learning-climate-change-problem/?sh=7120e1176b43>

80



HAI AI Index Report 2025 (April 2025), https://hai.stanford.edu/assets/files/hai_ai_index_report_2025.pdf

81

AI Homework

82

Next class, a ready-to-use GPU computer

- Cloud
 - Google Colab
- Local installation
 - NVIDIA driver, CUDA toolkit, cuDNN
 - <https://pytorch.org/get-started/locally/>
- NVIDIA's Docker containers
 - <https://docs.nvidia.com/deeplearning/frameworks/user-guide/index.html>



83

Thank You

84